9 Securing your microservices

This chapter covers

- Learning why security matters in a microservice environment
- Understanding OAuth2 and OpenID
- Setting up and configuring Keycloak
- Performing authentication and authorization with Keycloak
- Protecting your Spring microservice with Keycloak
- Propagating access tokens between services

Now that we have a robust microservices architecture, the task of covering security vulnerabilities becomes more and more essential. In this chapter, security and vulnerability go hand in hand. We'll define vulnerability as a weakness or flaw presented in an application. Of course, all systems have vulnerabilities, but the big difference lies in whether these vulnerabilities are exploited and cause harm.

Mentioning security often causes an involuntary groan from developers. Among developers, we hear comments such as, "It's obtuse, hard to understand, and even harder to debug." Yet, we won't find any developer (except for maybe an inexperienced developer) who says that they don't worry about security. Securing a microservices architecture is a complex and laborious task that involves multiple layers of protection, including these:

- *The application layer*—Ensures that the proper user controls are in place so that we can validate that a user is who they say they are and that they have permission to do what they're trying to do
- *Infrastructure*—Keeps the service running, patched, and up to date to minimize the risk of vulnerabilities
- *A network layer*—Implements network access controls so that a service is only accessible through well-defined ports and only to a small number of authorized servers

This chapter only covers how to authenticate and authorize users in our application layer (the first bullet point in the list). The other two items are extremely broad security top-

ics that are outside the scope of this book. As well, there are other tools, such as the OWASP Dependency-Check Project, that can help identify vulnerabilities.

NOTE The OWASP Dependency-Check Project is an OWASP Software Composition Analysis (SCA) tool that allows us to identify publicly disclosed vulnerabilities. If you want to find out more, we highly recommend you look at **https://owasp.org/www-project-dependency-check/**.

To implement authorization and authentication controls, we'll use the Spring Cloud Security module and Keycloak to secure our Spring-based services. Keycloak is open source identity and access management software for modern applications and services. This open source software is written in Java, and it supports SAML (Security Assertion Markup Language) v2 and OpenID Connect (OIDC)/OAuth2-federated identity protocols.

9.1 What is OAuth2?

OAuth2 is a token-based security framework that describes patterns for granting authorization but does not define how to actually perform authentication. Instead, it allows users to authenticate themselves with a third-party authentication service, called an *identity provider* (IdP). If the user successfully authenticates, they are presented with a token that must be sent with every request. The token can then be validated back to the authentication service.

The main objecive behind OAuth2 is that when multiple services are called to fulfill a user's request, the user can be authenticated by each service without having to present their credentials to each service processing their request. OAuth2 allows us to protect our REST-based services across different scenarios through authentication schemes called *grants*. The OAuth2 specification has four types of grants:

- Password
- Client credential
- Authorization code
- Implicit

We aren't going to walk through each of these grant types or provide code examples for each. That's too much material to cover in one chapter. Instead, we'll do the following:

- Discuss how our microservices can use OAuth2 through one of the simpler OAuth2 grant types, the password grant type
- Use JSON Web Tokens (JWTs) to provide a more robust OAuth2 solution and to establish a standard for encoding information in an OAuth2 token

- Walk through other security considerations that need to be taken into account when building microservices

NOTE We provide an overview of the other OAuth2 grant types in appendix B. If you're interested in diving into more detail on the OAuth2 spec and how to implement all the grant types, we highly recommend Justin Richer and Antonio Sanso's book, *OAuth2 in Action* (Manning, 2017), which is a comprehensive explanation of OAuth2.

The real power behind OAuth2 is that it allows application developers to easily integrate with third-party cloud providers and authenticate and authorize users with those services without having to pass the user's credentials continually to the third-party service.

OpenID Connect (OIDC) is a layer on top of the OAuth2 framework that provides authentication and profile information about who is logged in to the application (the identity). When an authorization server supports OIDC, it is sometimes called an *identity provider*. Before we get into the technical details of protecting our services, let's walk through the Keycloak architecture.

9.2 Introduction to Keycloak

Keycloak is an open source identity and access management solution for our services and applications. The main objective of Keycloak is to facilitate the protection of the services and applications with little or no code. Some characteristics of Keycloak include the following:

- It centralizes authentication and enables single sign-on (SSO) authentication.
- It allows developers to focus on business functionality instead of worrying about security aspects like authorization and authentication.
- It allows two-factor authentication.
- It is LDAP compliant.
- It offers several adapters to secure applications and servers easily.
- It lets you customize password policies.

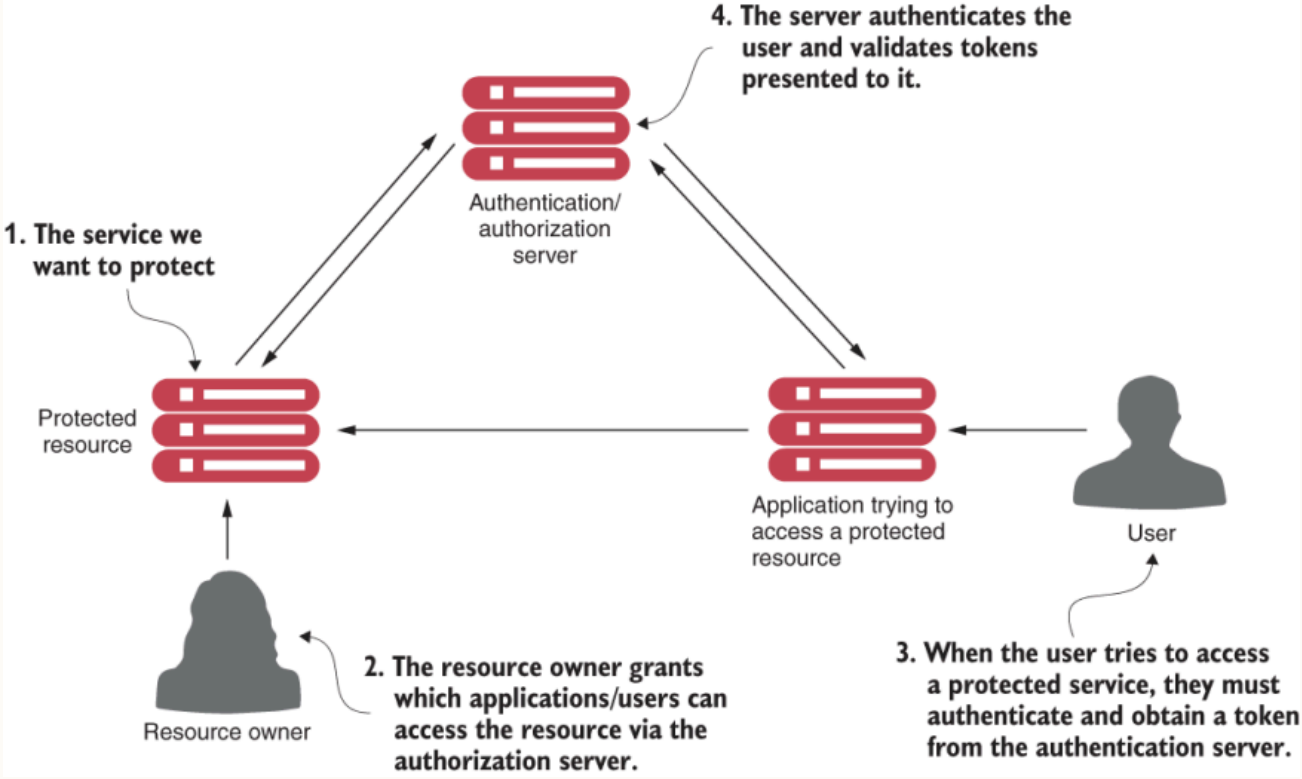


Figure 9.1 Keycloak allows a user to authenticate without having to present credentials constantly.

Keycloak security can be broken down into these four components: protected resource, resource owner, application, and authentication/authorization server. Figure 9.1 shows how the four components interact together.

- *A protected resource*—The resource (in our case, a microservice) you want to protect, ensuring that only authenticated users who have the proper authorization can access it.
- *A resource owner*—This owner defines which applications are allowed to call the service, which users are given access to the service, and what the users can do with the service. Each application registered by the resource owner is given an application name that identifies the application, along with a secret key. The combination of the application name and the secret key are part of the credentials that are passed when authenticating an access token.
- *An application*—This is the application that's going to call the service on behalf of a user. After all, users rarely invoke a service directly. Instead, they rely on an application to do the work for them.
- *Authentication/authorization server*—The authentication server is the intermediary between the application and the services being consumed. The authentication server allows the user to authenticate themselves without having to pass their user credentials to every service the application is going to call on their behalf.

As previously mentioned, the Keycloak security components interact together to authenticate the service users. Users authenticate with the Keycloak server by providing their credentials and the application/device they're using to access a protected resource (the microservice). If the users' credentials are valid, the Keycloak server provides an authentication token that can be passed from service to service each time a service is used by the users.

The protected resource can then contact the Keycloak server to determine the token's validity and to retrieve assigned roles for a user. Roles are used to group related users together and to define which resources they can access. For this chapter, we'll use Keycloak roles to determine what HTTP verbs a user can use to call on which authorized service endpoints.

Web service security is an extremely complicated subject. We need to understand who's going to call our services (internal users or external users), how they're going to call our service (internal web-based client, mobile device, or web application), and what actions they're going to take with our code.

On authentication vs. authorization

We've often found that developers "mix and match" the meaning of the terms authentication and authorization. Authentication is the act of a user proving who they are by providing credentials. Authorization determines whether a user is allowed to do what they want to do. For instance, user Illary could prove her identity by providing a user ID and password, but she may not be authorized to look at sensitive data (payroll data, for example). For our discussion, a user must be authenticated before authorization takes place.

9.3 Starting small: Using Spring and Keycloak to protect a single endpoint

To understand how to set up the authentication and authorization pieces, we'll do the following:

- Add a Keycloak service to Docker.
- Set up a Keycloak service and register the O-stock application as an authorized application that can authenticate and authorize user identities.
- Use Spring Security to protect our O-stock services. We're not going to build a UI for O-stock; instead, we'll simulate a user logging in to Postman to provide authentication for our Keycloak service.

- Protect the licensing and organization services so that they can only be called by an authenticated user.

13.1 Adding Keycloak to Docker

This section explains how to add the Keycloak service to our Docker environment. To achieve this, let's start by adding the code shown in the following listing to our docker-compose.yml file.

NOTE In case you didn't follow the previous chapter's code listings, you can download the code created in chapter 8 from the following link: https://github.com/ihuaylupo/manning-smia/tree/master/chapter8.

Listing 9.1 Adding a Keycloack service in docker-compose.yml

```
//Part of docker-compose.yml omitted for conciseness
...

keycloak:                    (1)
  image: jboss/keycloak
  restart: always
  environment:
    KEYCLOAK_USER: admin     (2)
    KEYCLOAK_PASSWORD: admin (3)
  ports:
    - "8080:8080"
  networks:
    backend:
      aliases:
        - "keycloak"
```

(1) Keycloak Docker service name

(2) Keycloak Admin Console's username

(3) Keycloak Admin Console's password

NOTE You can use Keycloak with several databases like H2, PostgreSQL, MySQL, Microsoft SQL Server, Oracle, and MariaDB. For the examples in this chapter, we'll use the default embedded H2 database. We highly recommend visiting the following link if you want to use another database: https://github.com/keycloak/keycloak-containers/tree/master/docker-compose-examples.

Notice that in the code in listing 9.1, we used port 8080 for Keycloak. If we go back a few chapters, we also exposed the licensing service in that port. We will map the licensing service in the docker-compose.yml file to port 8180 instead of 8080 to make everything work, but in the meantime, the following code shows you how to change that port number:

```
licensingservice:
  image: ostock/licensing-service:0.0.3-SNAPSHOT
  ports:
    - "8180:8080"
```

NOTE To make Keycloak work in our local environment, we need to add the host entry 127.0.0.1 Keycloak to our hosts file. If you are using Windows, you need to add the host entry at C:\Windows\System32\drivers\etc\hosts, and if you are using Linux, the host file is at /etc/hosts. Why do we need that entry? The containers can talk to each other using the network aliases or MAC addresses, but our Postman needs to use localhost to invoke the services.

3.2 Setting up Keycloak

Now that we have Keycloak in the docker-compose.yml, let's run the following command. We'll do this at the root folder of our code:

```
docker-compose -f docker/docker-compose.yml up
```

Once the services are up, let's visit the following link to open the Keycloak Administration Console: **http://keycloak:8080/auth/**. Configuring Keycloak is a straightforward process. The first time we access Keycloak, a Welcome page is displayed. This page shows different options, such as visiting the Administration Console, documentation, reporting issues, and more. In our case, we want to select Administration Console. Figure 9.2 shows the Welcome page.

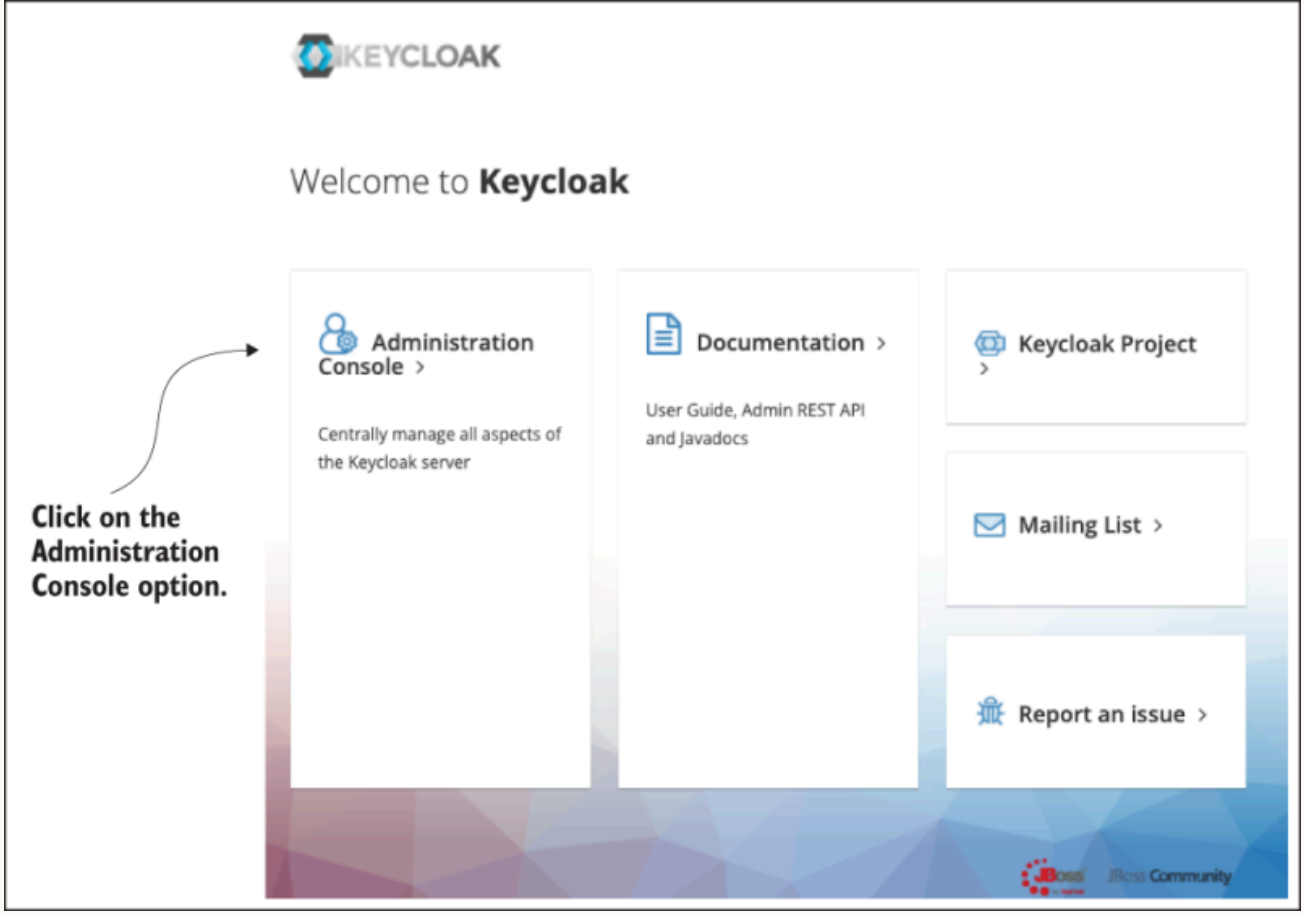


Figure 9.2 The Keycloak Welcome page

The next step is to enter the username and password that we previously defined in the docker-compose.yml file. Figure 9.3 shows this step.

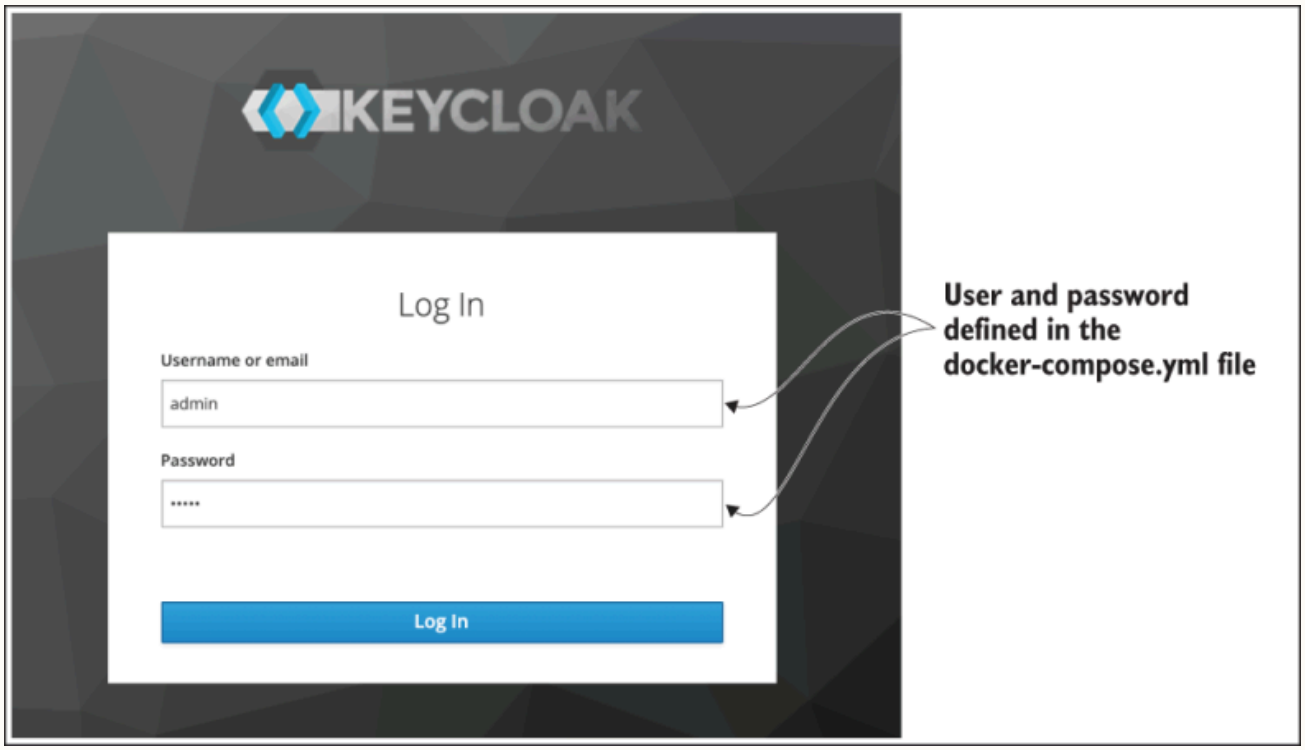


Figure 9.3 The Keycloak Log In page

To continue configuring our data, let's create our realm. A *realm* is a concept that Keycloak uses to refer to an object that manages a set of users, credentials, roles, and groups. To create our realm, click the Add Realm option that is shown in the Master drop-down menu after we log in to Keycloak. We'll call this realm spmia-realm.

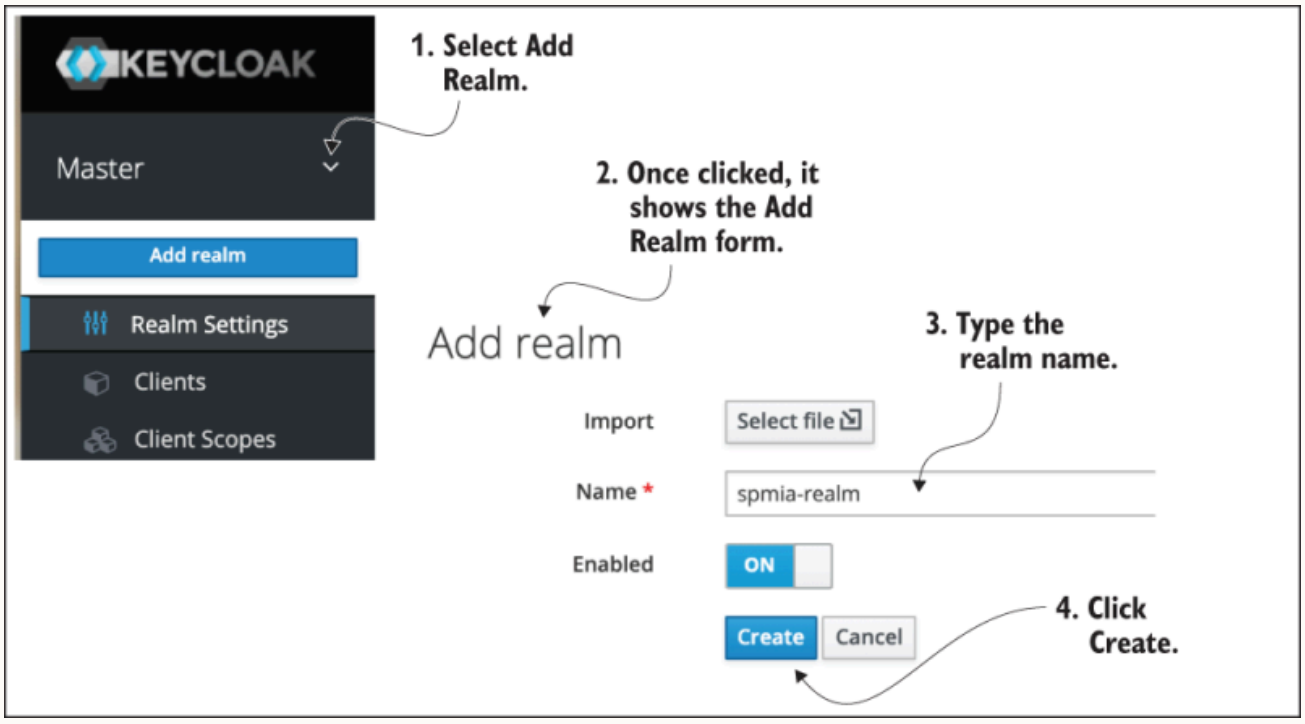


Figure 9.4 The Keycloak Add Realm page displays a form that allows the user to enter the realm name.

Figure 9.4 shows how to create the spmia-realm in our Keycloak service.

Once the realm is created, you will see the main page for spmia-realm with the configuration shown in figure 9.5.

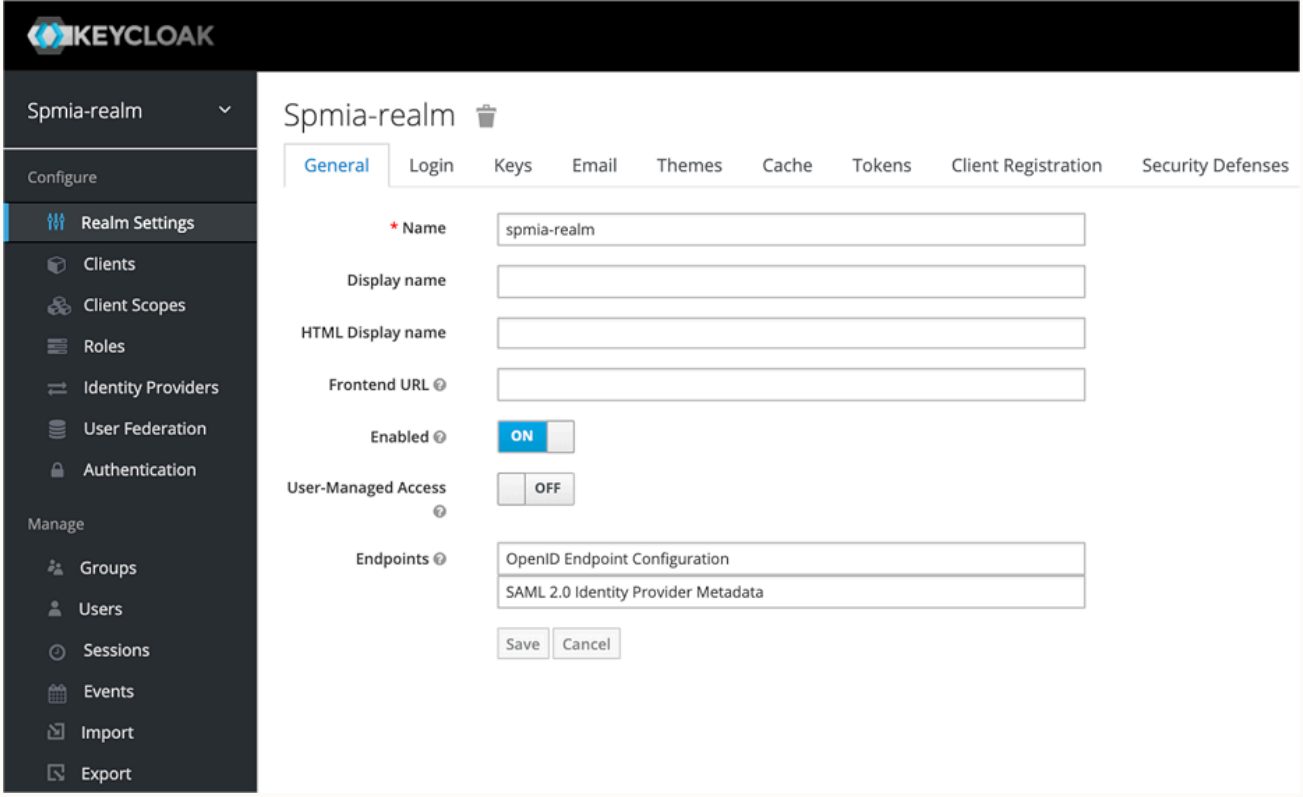


Figure 9.5 Our Keycloak Spmia-realm configuration page

3.3 Registering a client application

The next step in our configuration is to create a client. Clients in Keycloak are entities that can request user authentication. The clients are often the applications or services that we want to secure by providing a single sign-on (SSO) solution. To create a client, let's click the Clients option on the left menu. Once clicked, you will see the page shown in figure 9.6.

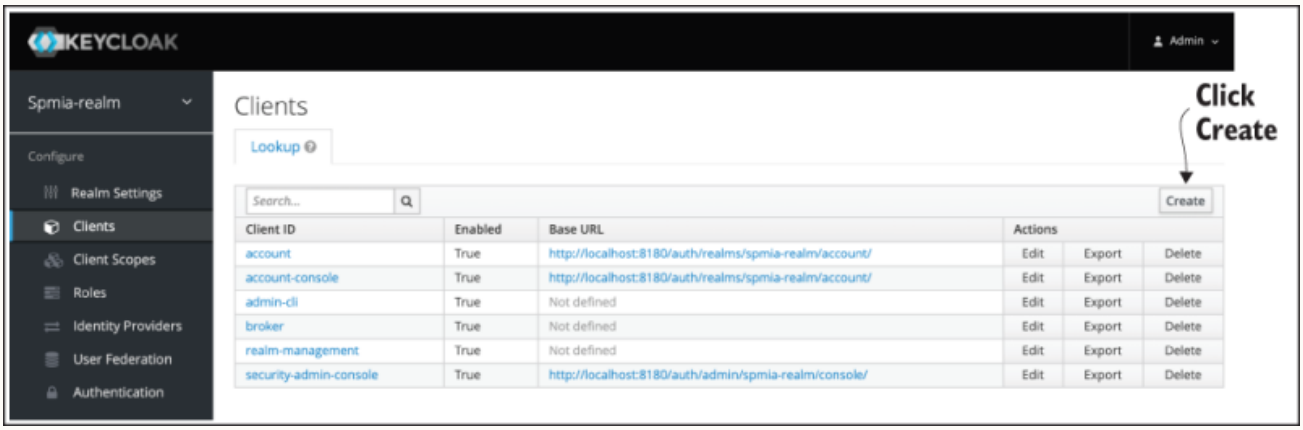


Figure 9.6 O-stock's Spmia-realm Clients page

Once the Client's list is shown, click the Create button (displayed in the table's top-right corner as shown in the previous figure). Once clicked, you will see an Add Client form that asks for the following information:

- Client ID
- Client Protocol
- Root URL

Enter the information as shown in figure 9.7.

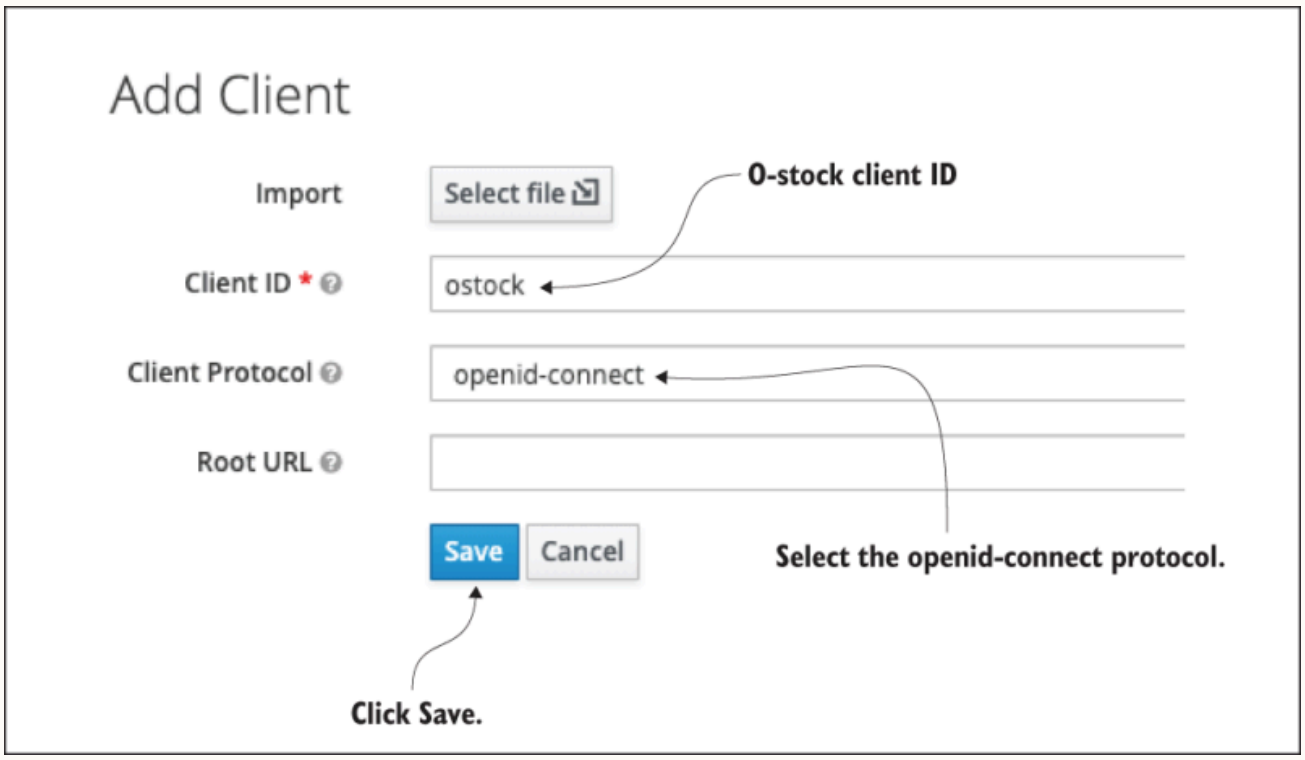


Figure 9.7 O-stock's Keycloak client information

After saving the client, you will be presented with the Client Configuration page shown in figure 9.8. On that page, we'll enter the following data:

- Access Type: Confidential
- Service Accounts Enabled: On
- Authorization Enabled: On
- Valid Redirect URLs: http://localhost:80*
- Web Origins: *

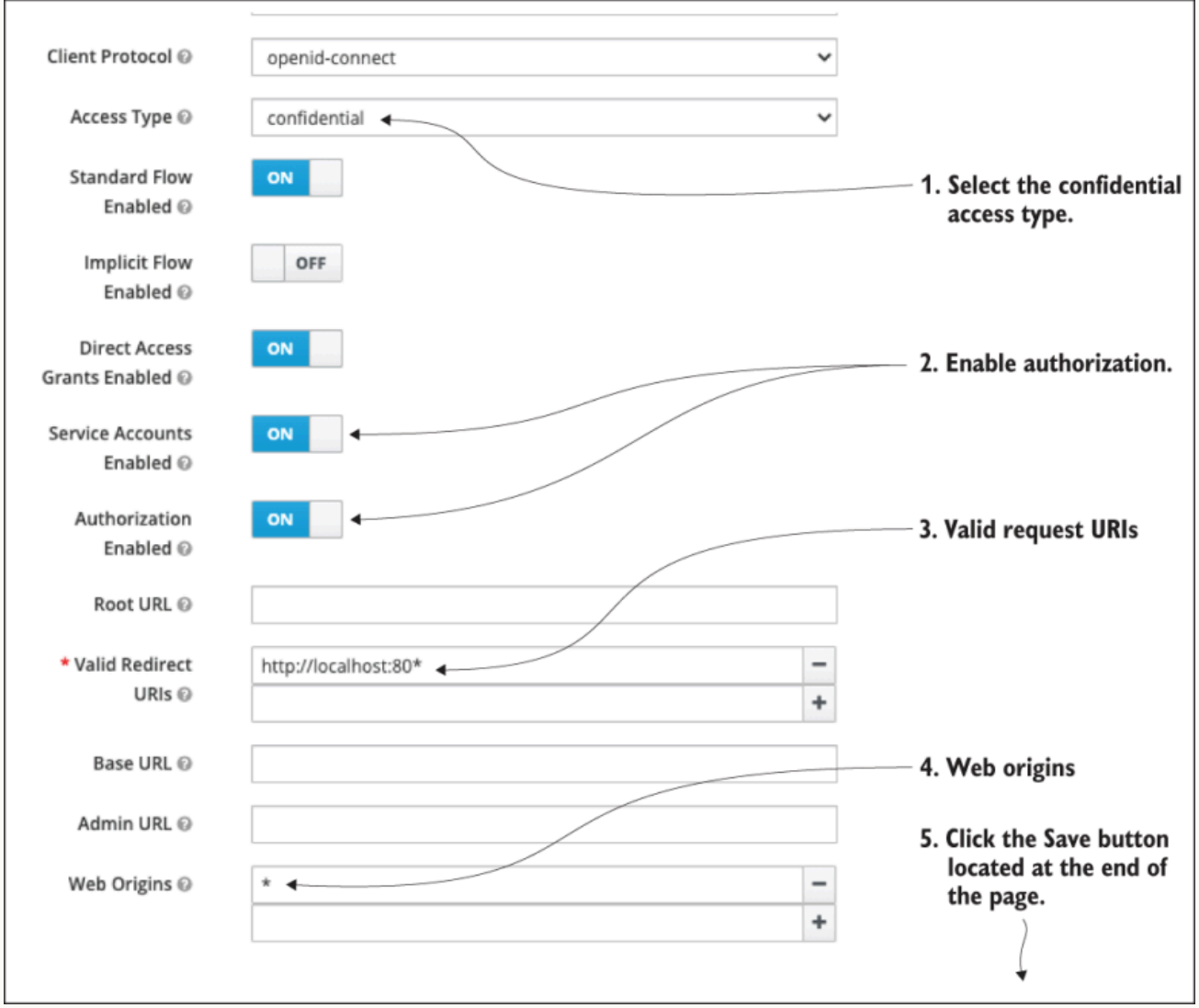


Figure 9.8 Additional configuration for O-stock's Keycloak client

For this example, we only created a global client called ostock, but here's where you can configure as many clients as you want.

The next step is to set up the client roles, so let's click the Roles tab. To better understand client roles, let's imagine that our application will have two types of users: admins and regular users. The admin users can execute all of the application services, and the regular users are only going to be allowed to execute some of the services.

Once the Roles page is loaded, you will see a list of predefined client roles. Let's click the Add Role button displayed in the top-right corner of the Roles table. Once clicked, you will see the Add Role form shown in figure 9.9.

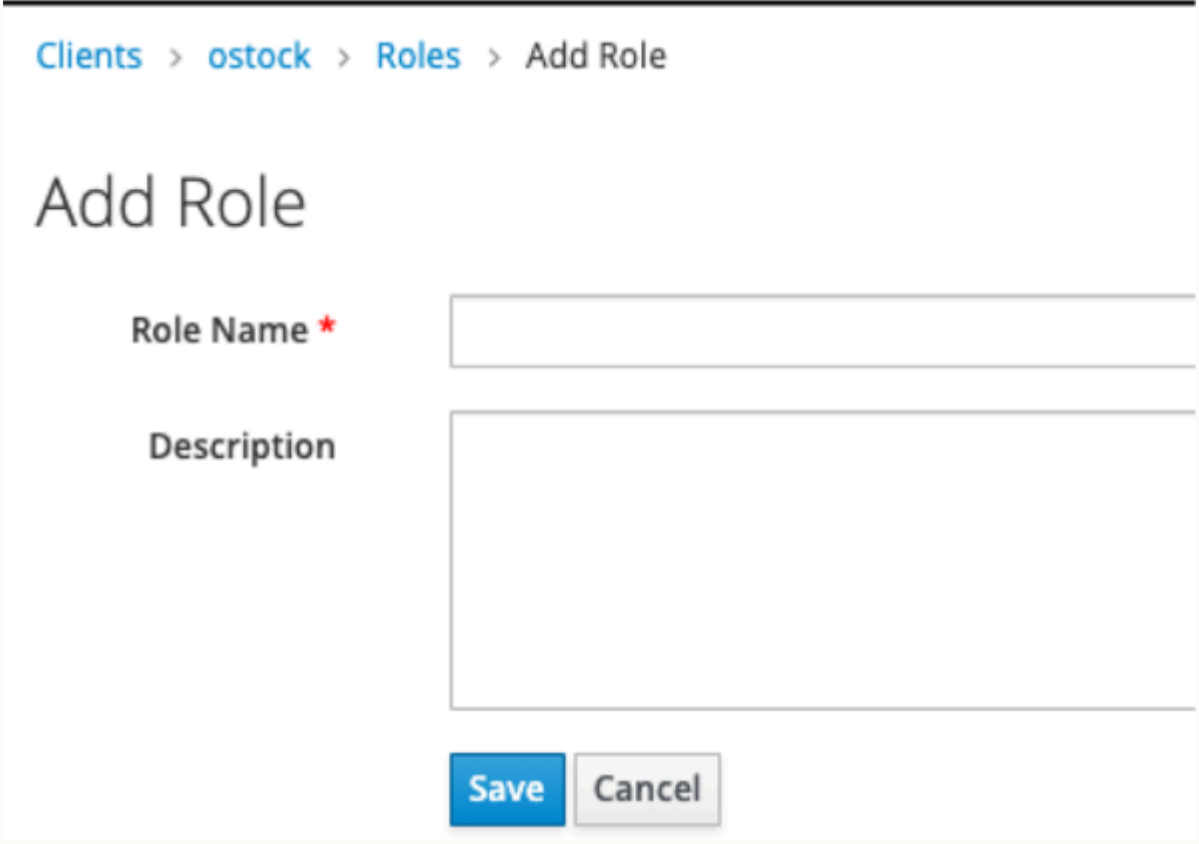


Figure 9.9 O-stock's Keycloak Add Role page

On the Add Role page, we need to create the following client roles. In the end, you will have a list similar to the one shown in figure 9.10.

- USER
- ADMIN

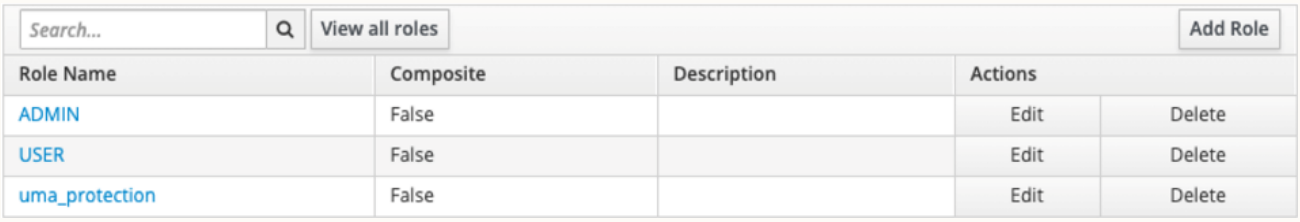


Figure 9.10 O-stock's Keycloak client roles: USER and ADMIN

Now that we have finished our basic client configuration, let's visit the Credentials page. The Credentials page will show the required client secret for the authentication process. Figure 9.11 shows how the Credentials page looks.

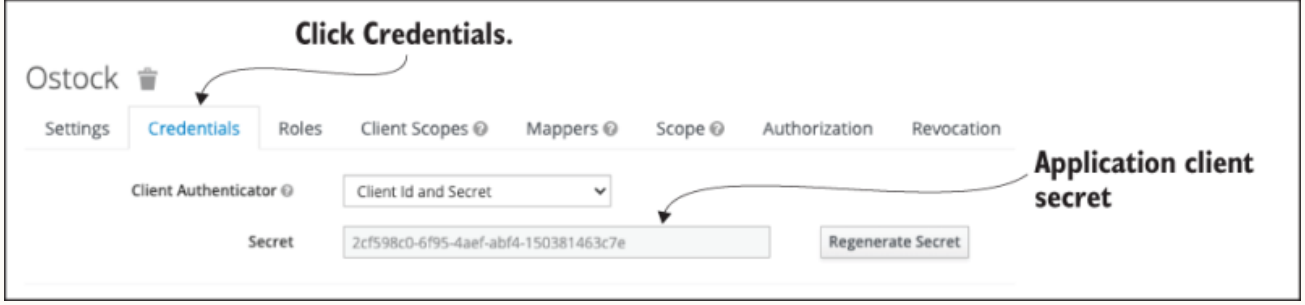


Figure 9.11 O-stock's Keycloak client secret on the Credentials page

The next step in our configuration is to create realm roles. The realm roles will allow us to have better control of what roles are being set for each user. This is an optional step. If you don't want to create these roles, you can go ahead and create the users directly. But later on, it might be harder to identify and maintain the roles for each user.

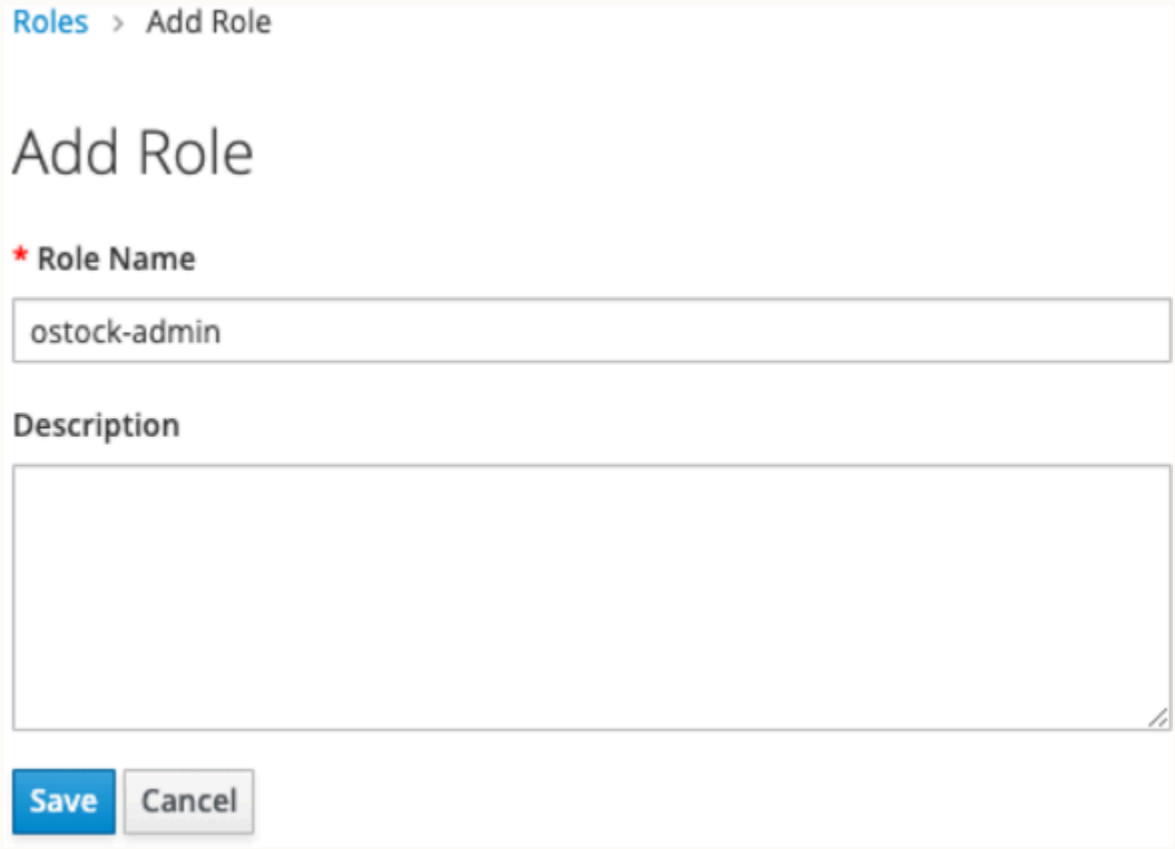


Figure 9.12 Creating the ostock-admin realm role

To create the realm roles, let's click the Roles option shown in the left menu and then click the Add Role button at the table's top right. As with the client roles, we will create two types of realm roles: the ostock-user and the ostock-admin. Figures 9.12 and 9.13 show you how to create the ostock-admin realm role.

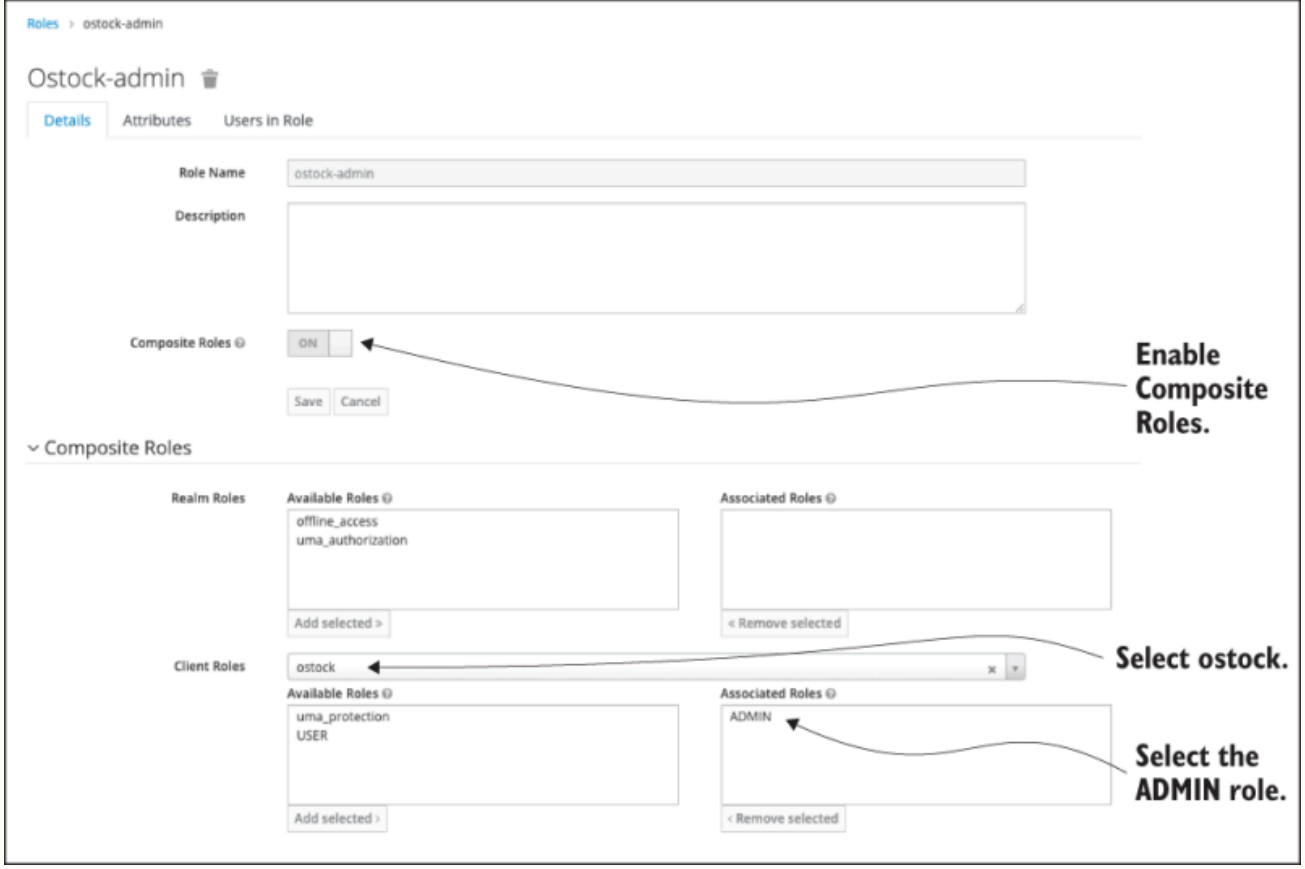


Figure 9.13 Specifying additional configurations for the ostock-admin realm role

Now that we have the ostock-admin realm role configured, let's repeat the same steps to create the ostock-user role. Once you are done, you should have a list similar to the one shown in figure 9.14.



Figure 9.14 A list of O-stock's spmia-realm roles

9.3.4 Configuring O-stock users

Now that we've defined application-level and realm-level roles, names, and secrets, we're ready to set up individual user credentials and the roles for the users. To create the users, let's click the Users option shown in the left menu in Keycloak's admin console.

For the examples in this chapter, we will define two user accounts: illary.huaylupo and john.carnell. The john.carnell account will have the role of ostock-user and the illary.huaylupo account will have the role of ostock-admin. Figure 9.15 shows the Add User page. On this page, let's type the username and enable the user and the email verified options as shown in the figure.

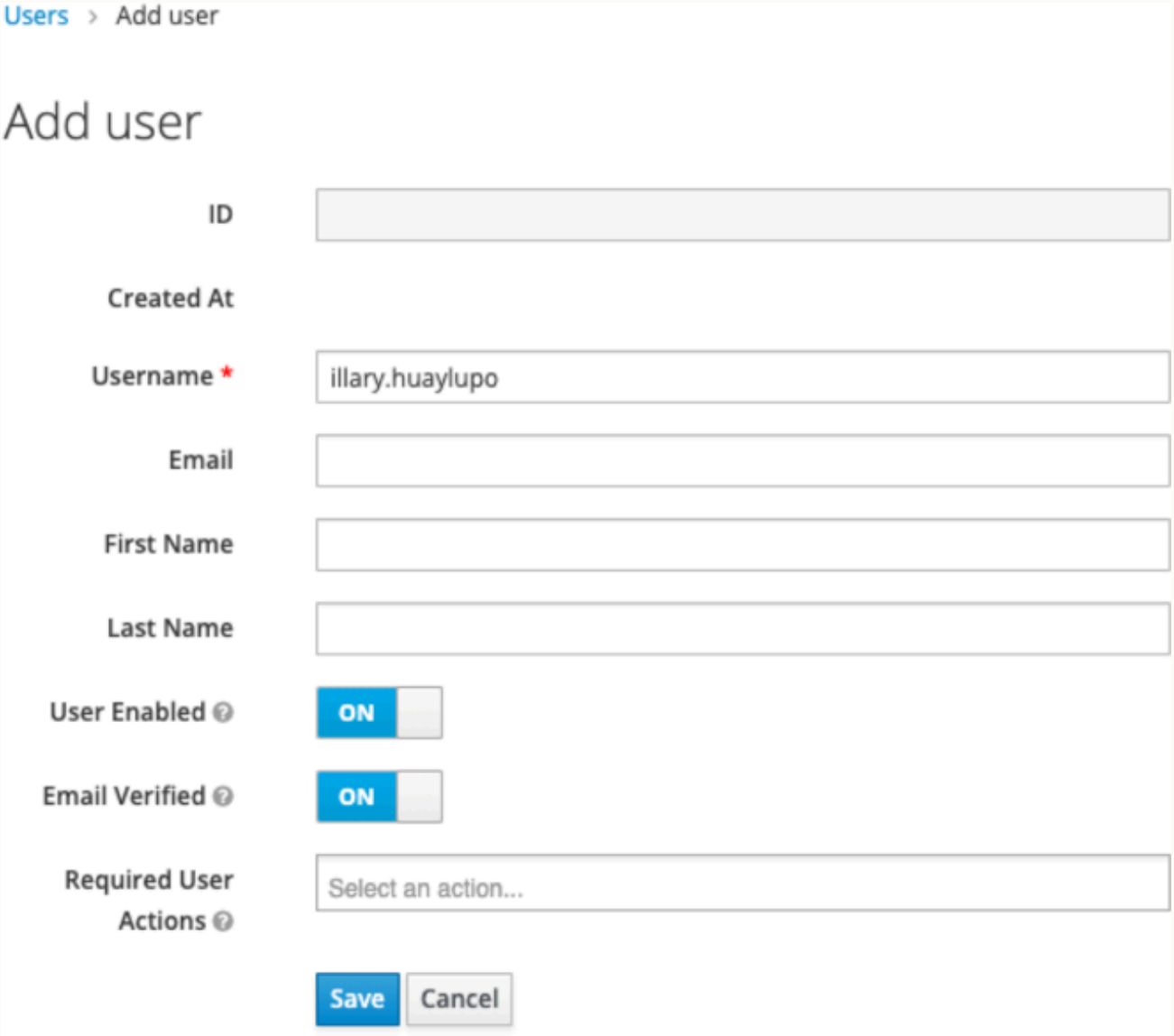


Figure 9.15 Keycloak's Add User page for O-stock's spmia-realm

NOTE Keycloak allows us to also add additional attributes to the users, such as first name, last name, email, address, birth date, phone number, and more. But, for this example, we will only set up the required attributes.

Once this form is saved, click the Credentials tab. You'll need to type the user's password, disable the Temporary option, and click the Set Password button. Figure 9.16 shows this step.

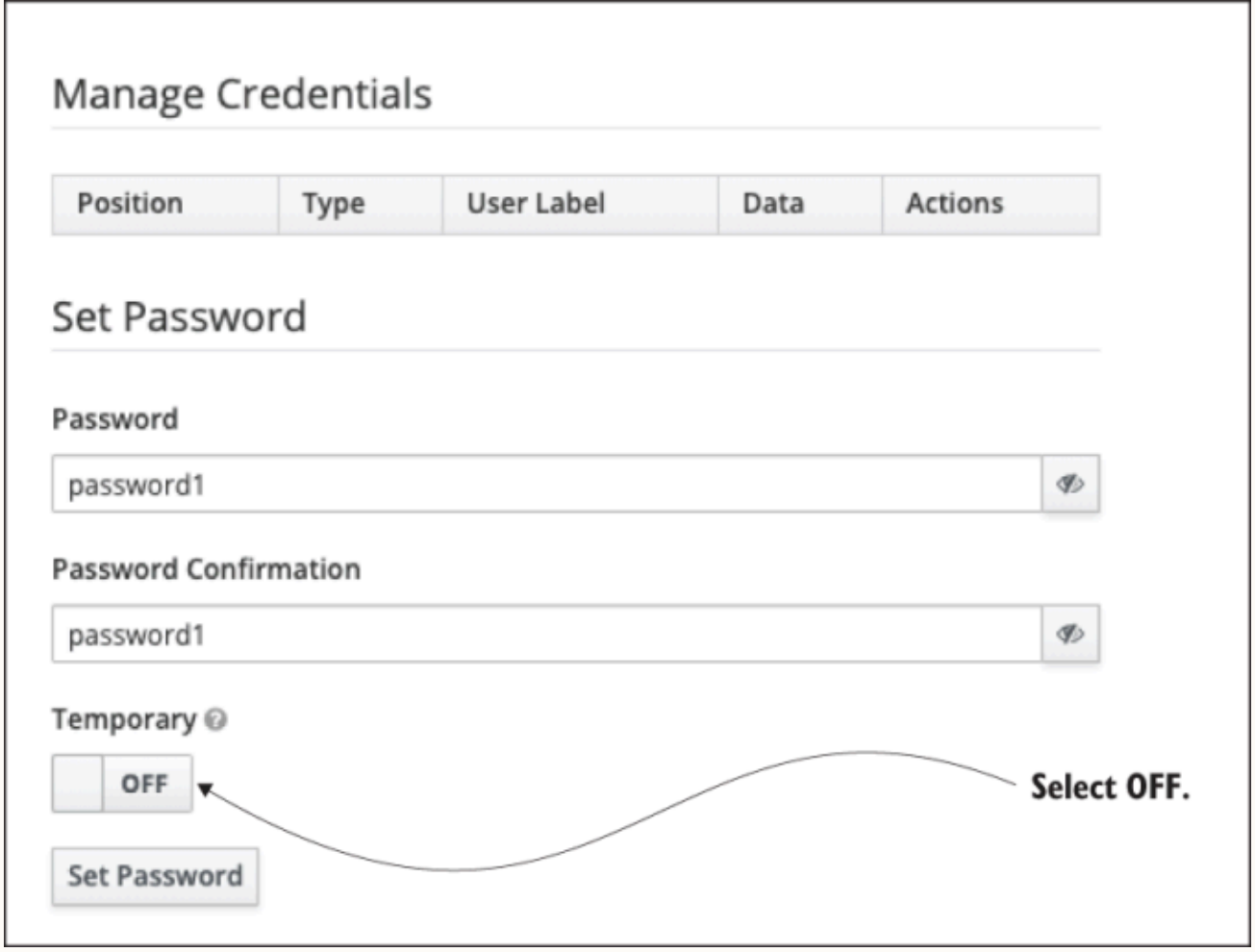


Figure 9.16 Setting the user's password and disabling the Temporary option for O-stock's user's credentials

Once the password is set, let's click the Role Mappings tab and assign the user a specific role. Figure 9.17 shows this step.

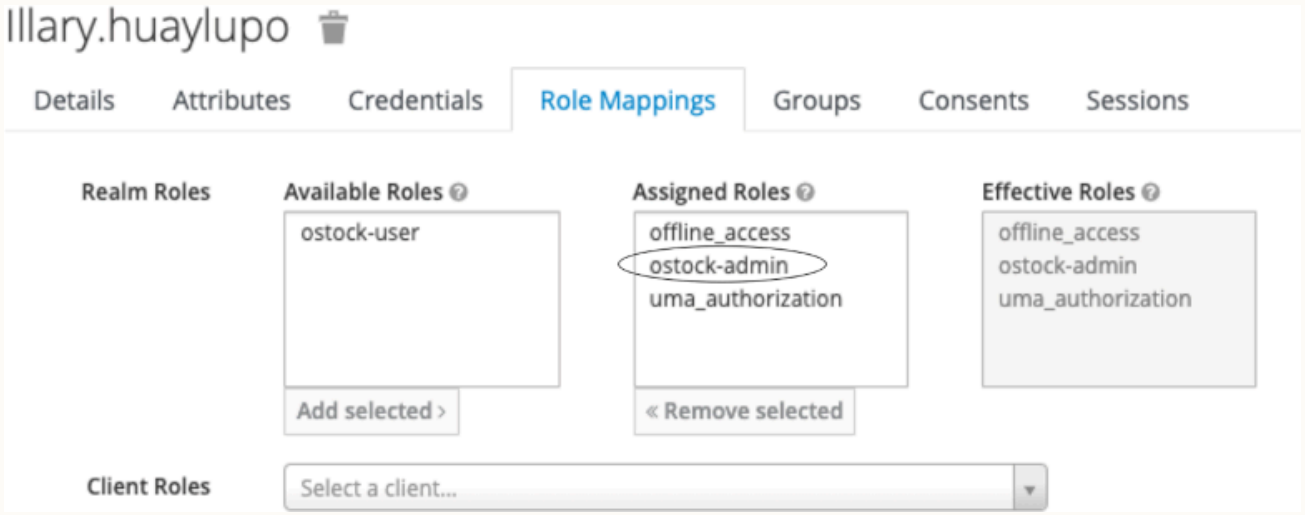


Figure 9.17 Mapping O-stock's realm roles to the created user

To finish our configuration, let's repeat the same steps for the other user, john.carnell in this example, assigning the ostock-user role to John.

9.3.5 Authenticating our O-stock users

At this point, we have enough of our base Keycloak server functionality in place to perform application and user authentication for the password grant flow. Let's start our authentication service. To do this, click the Realm Settings option in the left menu, and then click the OpenID Endpoint Configuration link to see a list of available endpoints for our realm. These steps are shown in figures 9.18 and 9.19.

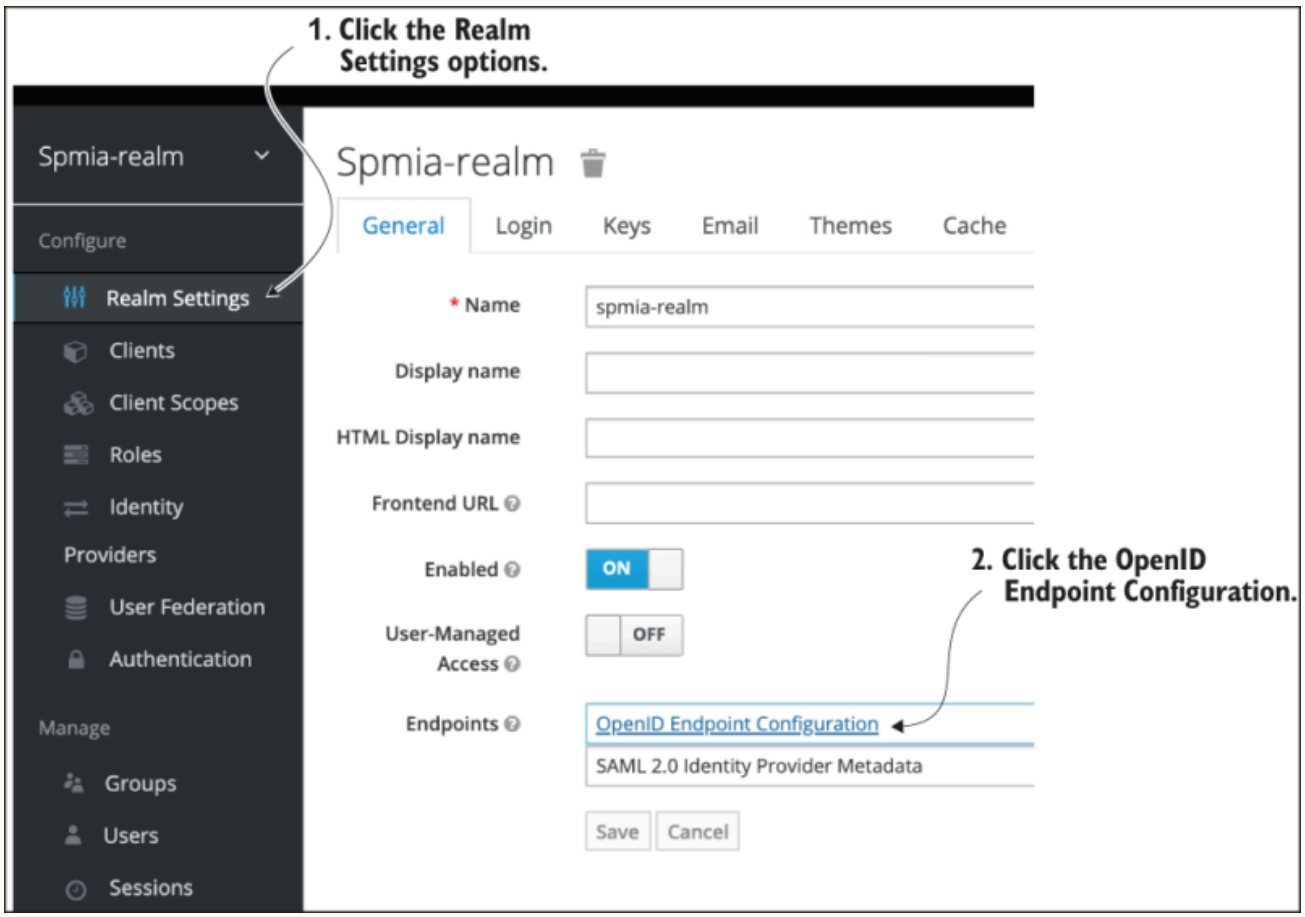


Figure 9.18 Selecting the OpenID Endpoint Configuration link for O-stock's Keycloak spmia-realm



Figure 9.19 Mapping O-stock's realm roles to the created user

Now we'll simulate a user that wants to acquire an access token. We'll do this by using Postman to POST to the endpoint `http://keycloak:8080/auth/realms/spmia-realm/protocol/openid-connect/token` and then provide the application, secret key, user ID, and password.

NOTE Remember, in this example, we use the 8080 port because it is the port we previously defined in the docker-compose.yml file for Keycloak.

To simulate a user acquiring an authentication token, we need to set up Postman with the application name and secret key. For this, we'll pass these elements to our authentication server endpoint using basic authentication. Figure 9.20 shows how Postman is set up to execute a basic authentication call. Note that we will use the application name we previously defined and the secret application key as a password:

```
Username: <CLIENT_APPLICATION_NAME>
Password: <CLIENT_APPLICATION_SECRET>
```

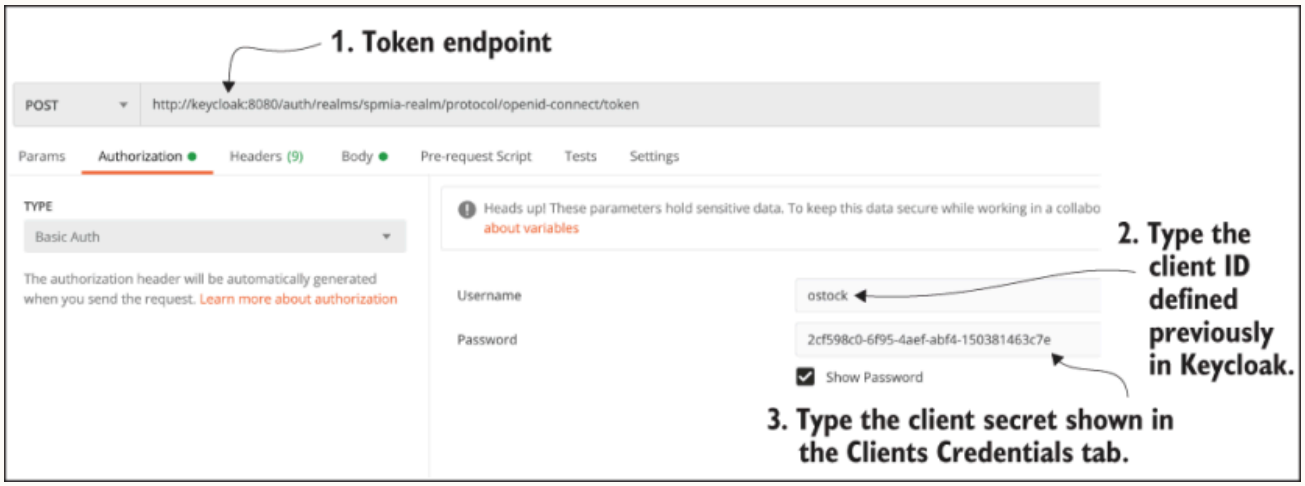


Figure 9.20 Setting up O-stock's basic authentication using the application key and secret

However, we're not ready to make the call to get the token just yet. Once the application name and secret key are configured, we need to pass in the following information to the service as HTTP form parameters:

- `grant_type` —The grant type to execute. In this example, we'll use a password grant.
- `username` —Name of the user logging in.
- `password` —Password of the user logging in.

Figure 9.21 shows how these HTTP form parameters are configured for our authentication call.

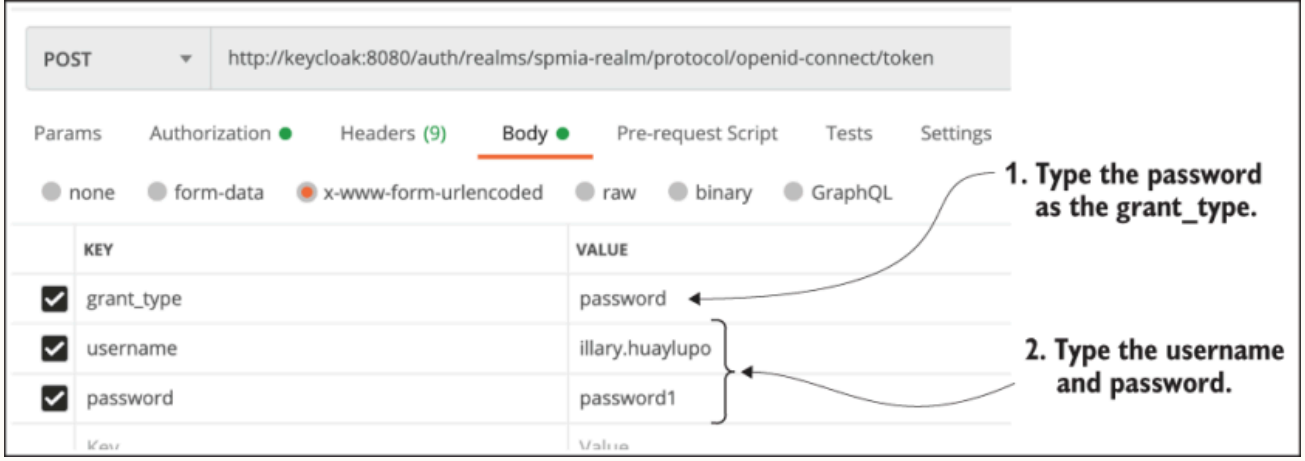


Figure 9.21 When requesting an access token, the user's credentials are passed in as HTTP form parameters to the `/openid-connect/token` endpoint.

Unlike other REST calls in this book, this list's parameters will not be passed in as a JSON body. The authentication standard expects all parameters passed to the token generation endpoint to be HTTP form parameters. Figure 9.22 shows the JSON payload that's returned from the `/openid-connect/token` call.

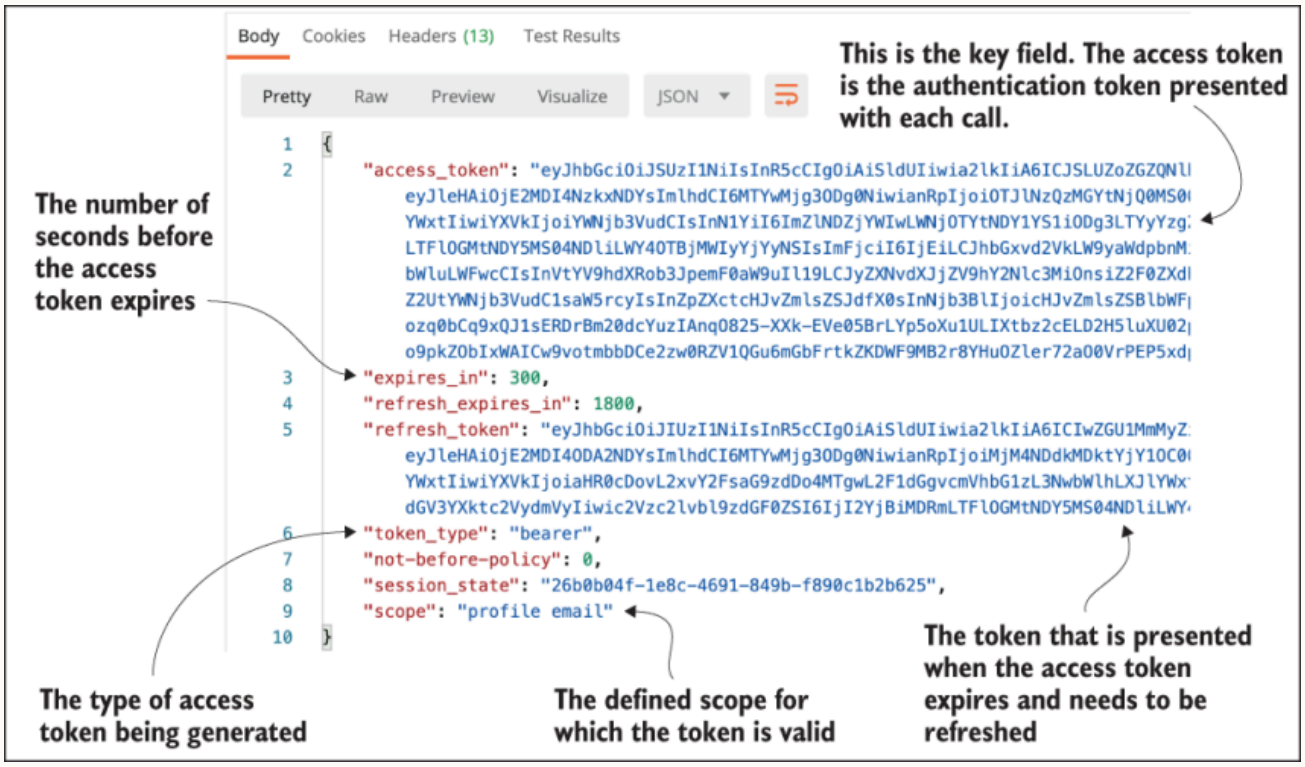


Figure 9.22 JSON payload returned after a successful client credential validation.

The JSON payload contains five attributes:

- `access_token` —The access token that will be presented with each service call the user makes to a protected resource.

- `token_type` —The authorization specification allows us to define multiple token types. The most common token type used is the Bearer Token. (We won't cover any of the other token types in this chapter.)
- `refresh_token` —The refresh token can be presented back to the authorization server to reissue a token after it expires.
- `expires_in` —This is the number of seconds before the access token expires. The default value for authorization token expiration in Spring is 12 hours.
- `scope` —This defines the scope for which this access token is valid.

Now that we have retrieved a valid access token from the authorization server, we can decode the JWT with **https://jwt.io** to retrieve all the access token information. Figure 9.23 shows the results of the decoded JWT.

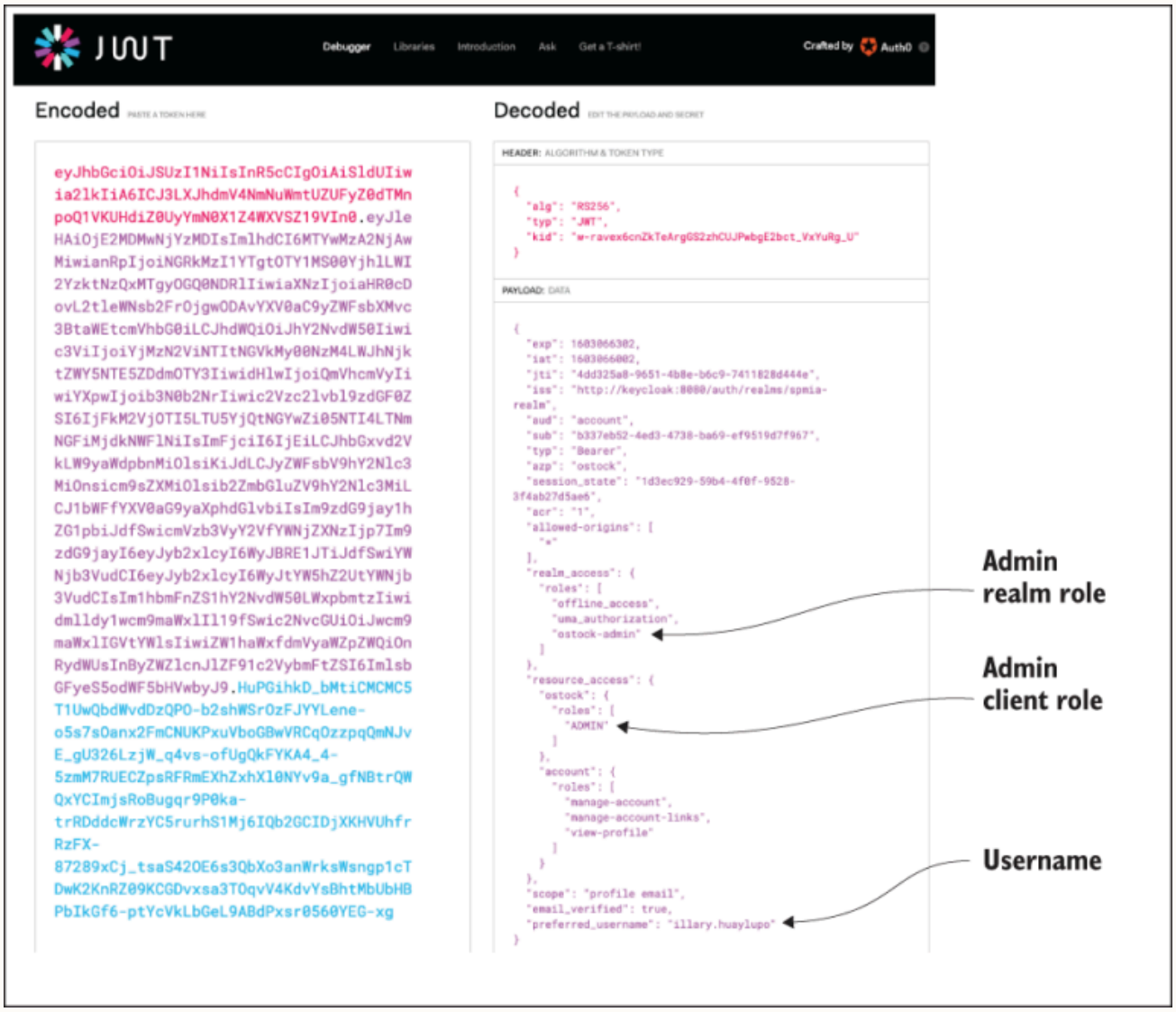


Figure 9.23 Looking up user information based on the issued access token

9.4 Protecting the organization service using Keycloak

Once we've registered a client in our Keycloak server and set up individual user accounts with roles, we can begin exploring how to protect a resource using Spring Security and

the Keycloak Spring Boot Adapter. While the creation and management of access tokens is the Keycloak server's responsibility, in Spring, the definition of which user roles have permissions to do what actions occurs at the individual service level. To set up a protected resource, we need to take the following actions:

- Add the appropriate Spring Security and Keycloak JARs to the service we're protecting.
- Configure the service to point to our Keycloak server.
- Define what and who can access the service.

Let's start with one of the simplest examples of setting up a protected resource. For that, we'll take our organization service and ensure that it can only be called by an authenticated user.

9.4.1 Adding the Spring Security and Keycloak JARs to the individual services

As usual with Spring microservices, we have to add a couple of dependencies to the organization service's Maven configuration file: organization-service/pom.xml. The following listing shows the new dependencies.

Listing 9.2 Configuring Keycloak and Spring security dependencies

```
//Part of pom.xml omitted for conciseness
    <dependency>
        <groupId>org.keycloak</groupId>
        <artifactId>
              keycloak-spring-boot-starter                    1
        </artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>
              spring-boot-starter-security                    2
        </artifactId>
    </dependency>
  </dependencies>
  <dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
```

```
            <scope>import</scope>
        </dependency>
        <dependency>
            <groupId>org.keycloak.bom</groupId>
            <artifactId>
              keycloak-adapter-bom                    ❸
            </artifactId>
            <version>11.0.2</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

❶ Keycloak Spring Boot dependency

❷ Spring Security starter dependency

❸ Keycloak Spring Boot dependency management

4.2 Configuring the service to point to our Keycloak server

Once we set up the organization service as a protected resource, every time a call is made to the service, the caller must include the authentication HTTP header containing a Bearer access token to the service. Our protected resource then has to call back to the Keycloak server to see if the token is valid. Listing 9.3 shows the required Keycloak configuration. You need to add this configuration to the organization service's application properties file located in the configuration server repository.

Listing 9.3 Keycloak configuration in the organization-service.properties file

```
//Some properties omitted for conciseness

keycloak.realm = spmia-realm                ❶
keycloak.auth-server-url =
      http://keycloak:8080/auth             ❷
keycloak.ssl-required = external
keycloak.resource = ostock                  ❸
keycloak.credentials.secret =
      5988f899-a5bf-4f76-b15f-f1cd0d2c81ba   ❹
keycloak.use-resource-role-mappings = true
keycloak.bearer-only = true
```

❶ The created realm name

❷ The Keycloak server URL Auth endpoint: http://<keycloak_server_url>/auth

❸ The created client ID

❹ The created client secret

NOTE To make the examples more straightforward in this book, we use the classpath repository. You'll find the configuration file in this file: /configserver/src/main/resources/config/organization-service.properties.

4.3 Defining who and what can access the service

We're now ready to begin defining the access control rules around the service. To define access control rules, we need to extend a `KeycloakWebSecurityConfigurerAdapter` class and override the following methods:

- `configure()`
- `configureGlobal()`
- `sessionAuthenticationStrategy()`
- `KeycloakConfigResolver()`

The organization service's `SecurityConfig` class is located in the /organizationservice/src/main/java/com/optimagrowth/organization/config/SecurityConfig.java file. The next listing shows the code for this class.

Listing 9.4 Extending `SecurityConfig`

```
//Some properties omitted for conciseness

@Configuration                                          ❶

@EnableWebSecurity                                      ❷
@EnableGlobalMethodSecurity(jsr250Enabled = true)       ❸

public class SecurityConfig extends
    KeycloakWebSecurityConfigurerAdapter {              ❹

  @Override
  protected void configure(HttpSecurity http)
      throws Exception {                                ❺
    super.configure(http);
```

```
        http.authorizeRequests()
            .anyRequest()
            .permitAll();
        http.csrf().disable();
    }

    @Autowired
    public void configureGlobal(
            AuthenticationManagerBuilder auth)                    ❻
            throws Exception {
     KeycloakAuthenticationProvider keycloakAuthenticationProvider =
            keycloakAuthenticationProvider();
     keycloakAuthenticationProvider.setGrantedAuthoritiesMapper(
            new SimpleAuthorityMapper());
     auth.authenticationProvider(keycloakAuthenticationProvider);
    }

    @Bean
    @Override
    protected SessionAuthenticationStrategy
            sessionAuthenticationStrategy() {                     ❼
        return new RegisterSessionAuthenticationStrategy(
            new SessionRegistryImpl());
    }

    @Bean
    public KeycloakConfigResolver
            KeycloakConfigResolver() {                            ❽
        return new KeycloakSpringBootConfigResolver();
    }
}
```

❶ The class must be marked with @Configuration.

❷ Applies the configuration to the global WebSecurity

❸ Enables @RoleAllowed

❹ Extends KeycloakWebSecurityConfigurerAdapter

❺ Registers the Keycloak authentication provider

❻ Defines the session authentication strategy

❼ Defines the session authentication strategy

❽ By default, the Spring Security Adapter looks for a keycloak.json file.

Access rules can range from coarse-grained (any authenticated user can access the entire service) to fine-grained (only the application with this role, but accessing this URL through a DELETE is allowed). We can't discuss every permutation of Spring Security's access control rules, but we can look at several of the more common examples. These examples include protecting a resource so that

- Only authenticated users can access a service URL.
- Only users with a specific role can access a service URL.

PROTECTING A SERVICE WITH AN AUTHENTICATED USER

The first thing we're going to do is to protect the organization service so that it can only be accessed by an authenticated user. The following listing shows how you can build this rule into the `SecurityConfig.java` class.

Listing 9.5 Restricting access to only authenticated users

```
package com.optimagrowth.organization.security;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import
➥ org.springframework.security.config.annotation.web.builders.HttpSecurity;

@Configuration
@EnableWebSecurity
@EnableGlobalMethodSecurity(jsr250Enabled = true)
public class SecurityConfig extends KeycloakWebSecurityConfigurerAdapter {

   @Override
   protected void configure(HttpSecurity http)
         throws Exception {                    ❶
      super.configure(http);
      http.authorizeRequests()
         .anyRequest().authenticated();
   }
}
```

❶ The HttpSecurity object passed into the method configures all access rules.

All access rules are defined inside the `configure()` method. We'll use the `HttpSecurity` class passed in by Spring to define our rules. In this example, we will restrict all access to any URL in the organization service to authenticated users only.

Suppose we were to access the organization service without an access token present in the HTTP header. In this case, we'd get a 401 HTTP response code and a message indicating that full authentication to the service is required. Figure 9.24 shows the output of a call to the organization service without the authentication HTTP header.



Figure 9.24 Calling the organization service without an access token results in a failed call.

Next, we'll call the organization service *with* an access token. (To generate an access token, see section 9.2.5.) We want to cut and paste the `access_token` value from the returned JSON call to the `/openid-connect/token` endpoint and use it in our call to the organization service. Remember, when we call the organization service, we need to set the authorization type to Bearer Token with the `access_token` value. Figure 9.25 shows the callout to the organization service, but this time with an access token passed to it.

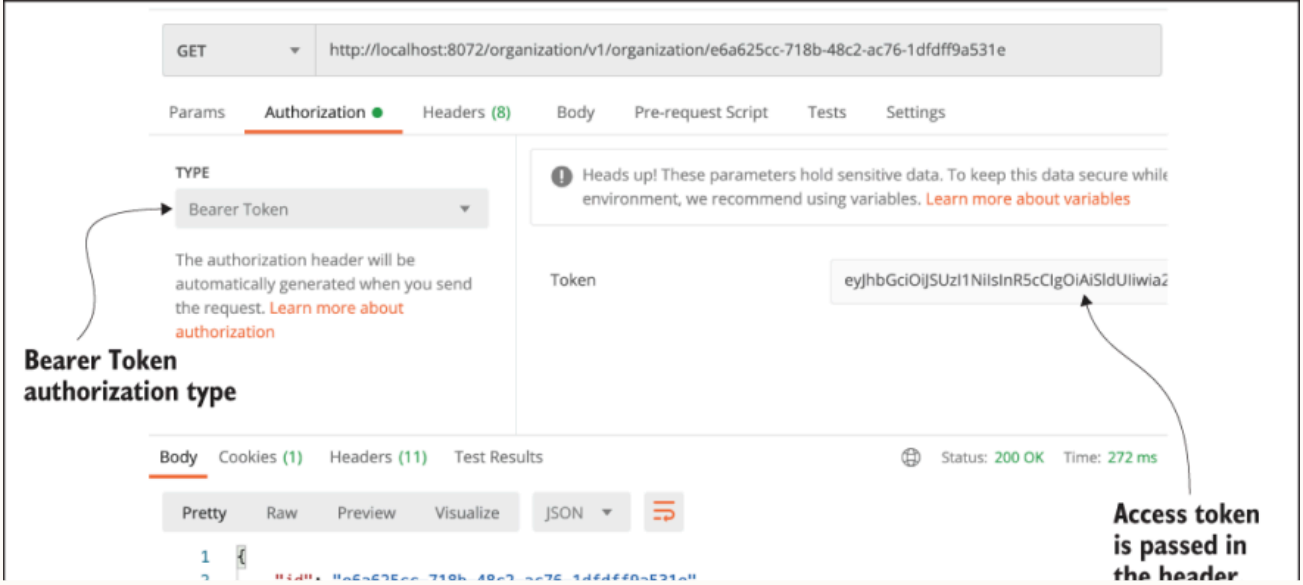


Figure 9.25 Passing in the access token on the call to the organization service

This is probably one of the most straightforward use cases for protecting an endpoint using JWTs. Next, we'll build on this example and restrict access to a specific endpoint for a specific role.

PROTECTING A SERVICE VIA A SPECIFIC ROLE

In the next example, we'll lock down the DELETE call on our organization service to only those users with ADMIN access. As you'll remember from section 9.2.4, we created two user accounts that can access O-stock services: illary.huaylupo and john.carnell. The john.carnell account had the role of USER assigned to it. The illary.huaylupo account had the USER role and the ADMIN role. We can permit specific roles to execute some methods by using `@RolesAllowed` in the controller. The following listing shows how.

Listing 9.6 Using the `@RolesAllowedAnnotation` in the OrganizationController.java

```
package com.optimagrowth.organization.security;
//Imports removed for conciseness

@RestController
@RequestMapping(value="v1/organization")
public class OrganizationController {

    @Autowired
    private OrganizationService service;
```

```
    @RolesAllowed({ "ADMIN", "USER" })                    (1)
    @RequestMapping(value="/{organizationId}",method = RequestMethod.GET)
    public ResponseEntity<Organization> getOrganization(
          @PathVariable("organizationId") String organizationId) {
       return ResponseEntity.ok(service.findById(organizationId));
    }

    @RolesAllowed({ "ADMIN", "USER" })                    (1)
    @RequestMapping(value="/{organizationId}",method = RequestMethod.PUT)
    public void updateOrganization( @PathVariable("organizationId")
            String id, @RequestBody Organization organization) {
       service.update(organization);
    }

    @RolesAllowed({ "ADMIN", "USER" })                    (1)
    @PostMapping
    public ResponseEntity<Organization>  saveOrganization(
             @RequestBody Organization organization) {
       return ResponseEntity.ok(service.create(organization));
    }

    @RolesAllowed("ADMIN")                                (2)
    @DeleteMapping(value="/{organizationId}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteLicense(@PathVariable("organizationId")
                              String organizationId) {
       service.delete(organizationId);
    }
}
```

(1) Indicates that only users with the USER and ADMIN roles can execute this action

(2) Indicates that only users with the ADMIN role can execute this action

Now, to obtain the token for john.carnell (`password: password1`), we need to execute the `openid-connect/token` POST request again. Once we have the new access token, we then need to call the DELETE endpoint for the organization service: http://localhost:8072/organization/v1/organization/dfd13002-57c5-47ce-a4c2-a1fda2f51513. We'll get a 403 Forbidden HTTP status code on the call and an error message indicating that access was denied for this service. The JSON text returned by our call is shown here:

```
{
   "timestamp": "2020-10-19T01:19:56.534+0000",
   "status": 403,
   "error": "Forbidden",
   "message": "Forbidden",
   "path": "/v1/organization/4d10ec24-141a-4980-be34-2ddb5e0458c7"
}
```

NOTE We use the 8072 port because it is the port we defined for the Spring Cloud Gateway in the previous chapter (specifically in the gateway-server.yml configuration file located in the Spring Cloud Configuration service repository).

If we tried the same call using the illary.huaylupo user account (password: password1) and its access token, we'd see a successful call that returns no content and the HTTP status code 204, No Content. At this point, we've looked at how to call and protect a single service (the organization service) with Keycloak. However, often in a microservice environment, we will have multiple service calls to carry out a single transaction. In these types of situations, we need to ensure that the access token is propagated from service call to service call.

9.4.4 Propagating the access token

To demonstrate propagating a token between services, we'll protect our licensing service with Keycloak as well. Remember, the licensing service calls the organization service to look up information. The question becomes, how do we propagate the token from one service to another?

We're going to set up a simple example where we'll have the licensing service call the organization service. If you followed the examples we built in the previous chapters, you'd see that both services are running behind a gateway. Figure 9.26 shows how an authenticated user's token will flow through the gateway, the licensing service, and then down to the organization service.

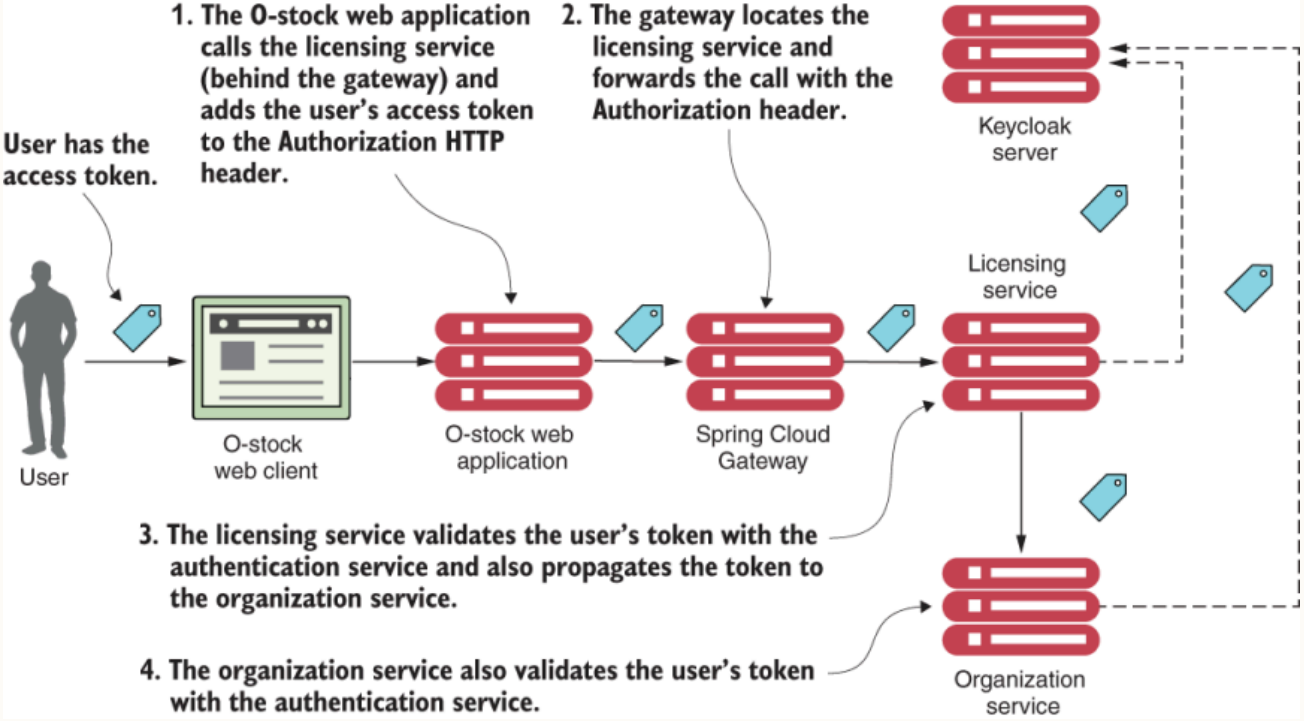


Figure 9.26 The access token must be carried throughout the entire call chain.

Here's what takes place in figure 9.26. Numbers corresponding to those in figure 9.26 are indicated in parentheses in the following.

- The user has already authenticated with the Keycloak server and places a call to the O-stock web application. The user's access token is stored in the user's session. The O-stock web application needs to retrieve some licensing data and call the licensing service REST endpoint (1). As part of the call to the licensing REST endpoint, the O-stock web application adds the access token via the HTTP `Authorization` header. The licensing service is only accessible behind a Spring Cloud Gateway.
- The gateway looks up the licensing service endpoint and then forwards the call to one of the licensing service's servers (2). The services gateway copies the authorization HTTP header from the incoming call and ensures that the HTTP header is forwarded on to the new endpoint.
- The licensing service receives the incoming call. Because the licensing service is a protected resource, the licensing service will validate the token with the Keycloak server (3) and then check the user's roles for the appropriate permissions. As part of its work, the licensing service invokes the organization service. When doing this, the licensing service needs to propagate the user's access token to the organization service.
- When the organization service receives the call, it takes the HTTP `Authorization` header and validates the token with the Keycloak server (4).

To implement these steps, we need to make several changes to our code. If we don't do this, we will get the following error while retrieving the organization information from

the licensing service:

```
message": "401 : {[status:401,
error: Unauthorized
message: Unauthorized,
path: /v1/organization/d898a142-de44-466c-8c88-9ceb2c2429d3}]
```

The first step is to modify the gateway so that it propagates the access token to the licensing service. By default, the gateway doesn't forward sensitive HTTP headers like cookie, set-cookie, and authorization to downstream services. To allow the propagation of the authorization HTTP header, we need to add to each route the following filter in the gateway-server.yml configuration file located in the Spring Cloud Config repository:

```
- RemoveRequestHeader= Cookie,Set-Cookie
```

This configuration is a blacklist of the sensitive headers that the gateway will keep from being propagated to a downstream service. The absence of the `Authorization` value in the `RemoveRequestHeader` list means that the gateway will allow that header through. If we don't set this configuration property, the gateway automatically blocks the propagation of all three values (`Cookie`, `Set-Cookie`, and `Authorization`).

Next, we need to configure our licensing service to include the Keycloak and Spring Security dependencies and set up any authorization rules we want for the service. Finally, we need to add the Keycloak properties to the application properties file in the configuration server.

CONFIGURING THE LICENSING SERVICE

When propagating access tokens, our first step is to add the Maven dependencies to our licensing service pom.xml file. The following listing shows the dependencies.

Listing 9.7 Configuring Keycloak and Spring Security dependencies

```
//Part of pom.xml omitted for conciseness
   <dependency>
      <groupId>org.keycloak</groupId>
      <artifactId>keycloak-spring-boot-starter</artifactId>
   </dependency>
   <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
  </dependencies>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-dependencies</artifactId>
        <version>${spring-cloud.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
      <dependency>
        <groupId>org.keycloak.bom</groupId>
        <artifactId>keycloak-adapter-bom</artifactId>
        <version>11.0.2</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
```

The next step is to protect the licensing service so that it can only be accessed by an authenticated user. The following listing shows the `SecurityConfig` class. You'll find the source code for this class in /licensing-service/src/main/java/com/optimagrowth/license/config/SecurityConfig.java.

Listing 9.8 Restricting access to authenticated users only

```
//Part of class declaration omitted for conciseness

@Configuration
@EnableWebSecurity
public class SecurityConfig extends KeycloakWebSecurityConfigurerAdapter {

 @Override
 protected void configure(HttpSecurity http) throws Exception {
    super.configure(http);
    http.authorizeRequests()
        .anyRequest().authenticated();
        http.csrf().disable();
 }
 @Override
 protected void configure(HttpSecurity http) throws Exception {
```

```
    super.configure(http);
    http.authorizeRequests()
        .anyRequest().authenticated();
    http.csrf().disable();
  }

  @Autowired
  public void configureGlobal(AuthenticationManagerBuilder auth)
                                  throws Exception {
    KeycloakAuthenticationProvider keycloakAuthenticationProvider =
                          keycloakAuthenticationProvider();
    keycloakAuthenticationProvider.setGrantedAuthoritiesMapper(new
                          SimpleAuthorityMapper());
    auth.authenticationProvider(keycloakAuthenticationProvider);
  }

  @Bean
  @Override
  protected SessionAuthenticationStrategy sessionAuthenticationStrategy() {
    return new RegisterSessionAuthenticationStrategy(
          new SessionRegistryImpl());
  }

  @Bean
  public KeycloakConfigResolver KeycloakConfigResolver() {
    return new KeycloakSpringBootConfigResolver();
  }
}
```

The final step in configuring the licensing service is to add the Keycloak configuration to our licensing-service.properties file. The following listing shows how this file should look.

Listing 9.9 Configuring Keycloak in licensing-service.properties

```
//Some properties omitted for conciseness

keycloak.realm = spmia-realm
keycloak.auth-server-url = http://keycloak:8080/auth
keycloak.ssl-required = external
keycloak.resource = ostock
keycloak.credentials.secret = 5988f899-a5bf-4f76-b15f-f1cd0d2c81ba
keycloak.use-resource-role-mappings = true
keycloak.bearer-only = true
```

Now that we have the gateway changes to propagate the authorization header and the licensing service set up, we can move on to our final step, which is to propagate the access token. All we need to do for this step is to modify how the code in the licensing service calls the organization service. For that, we need to ensure that the HTTP `Authorization` header is injected into the application call to the organization service.

Without Spring Security, we'd have to write a servlet filter to grab the HTTP header of the incoming licensing service call and then manually add it to every outbound service call in the licensing service. Keycloak provides a new REST template class that supports these calls. The class is called `KeycloakRestTemplate`. To use this class, we first need to expose it as a bean that can be autowired into a service calling another protected service. We'll do this in the /licensing-service/src/main/java/com/optimagrowth/license/config/SecurityConfig.java file by adding the code in the following listing.

Listing 9.10 Exposing `KeycloakRestTemplate` in `SecurityConfig`

```
package com.optimagrowth.license.service.client;
//Part of class definition omitted for conciseness

@ComponentScan(basePackageClasses = KeycloakSecurityComponents.class)
public class SecurityConfig extends KeycloakWebSecurityConfigurerAdapter {

   ...

   @Autowired
   public KeycloakClientRequestFactory keycloakClientRequestFactory;

   @Bean    #A
   @Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
   public KeycloakRestTemplate keycloakRestTemplate() {
      return new KeycloakRestTemplate(keycloakClientRequestFactory);
   }

   ...
}
```

To see the `KeycloakRestTemplate` class in action, we can look in the `OrganizationRestTemplateClient` class found in /licensing-service/src/main/java/com/optimagrowth/license/service/client/OrganizationRestTemplateClient.java. The following listing shows how `KeycloakRestTemplate` is autowired to this class.

Listing 9.11 Using `KeycloakRestTemplate` to propagate the access token

```
package com.optimagrowth.license.service.client;
//Imports removed for conciseness

@Component
public class OrganizationRestTemplateClient {
  @Autowired
  KeycloakRestTemplate restTemplate;                    (1)

  public Organization getOrganization(String organizationId){
    ResponseEntity<Organization> restExchange =          (2)
     restTemplate.exchange("http://gateway:8072/organization/
                  v1/organization/{organizationId}",
                HttpMethod.GET,
                null, Organization.class, organizationId);

    return restExchange.getBody();
  }
}
```

(1) KeycloakRestTemplate is a drop-in replacement for the standard RestTemplate. It handles the propagation of the access token.

(2) The invocation of the organization service is done in the exact same manner as a standard RestTemplate. Here, we point to the gateway server.

To test this code, you can request a service in the licensing service that calls the organization service to retrieve the data. For example, the following service retrieves the data of a specific license and then gets the associated information for the organization. Figure 9.27 shows the output of this call:

```
http://localhost:8072/license/v1/organization/d898a142-de44-466c-8c88-
➥ 9ceb2c2429d3/license/f2a9c9d4-d2c0-44fa-97fe-724d77173c62
```

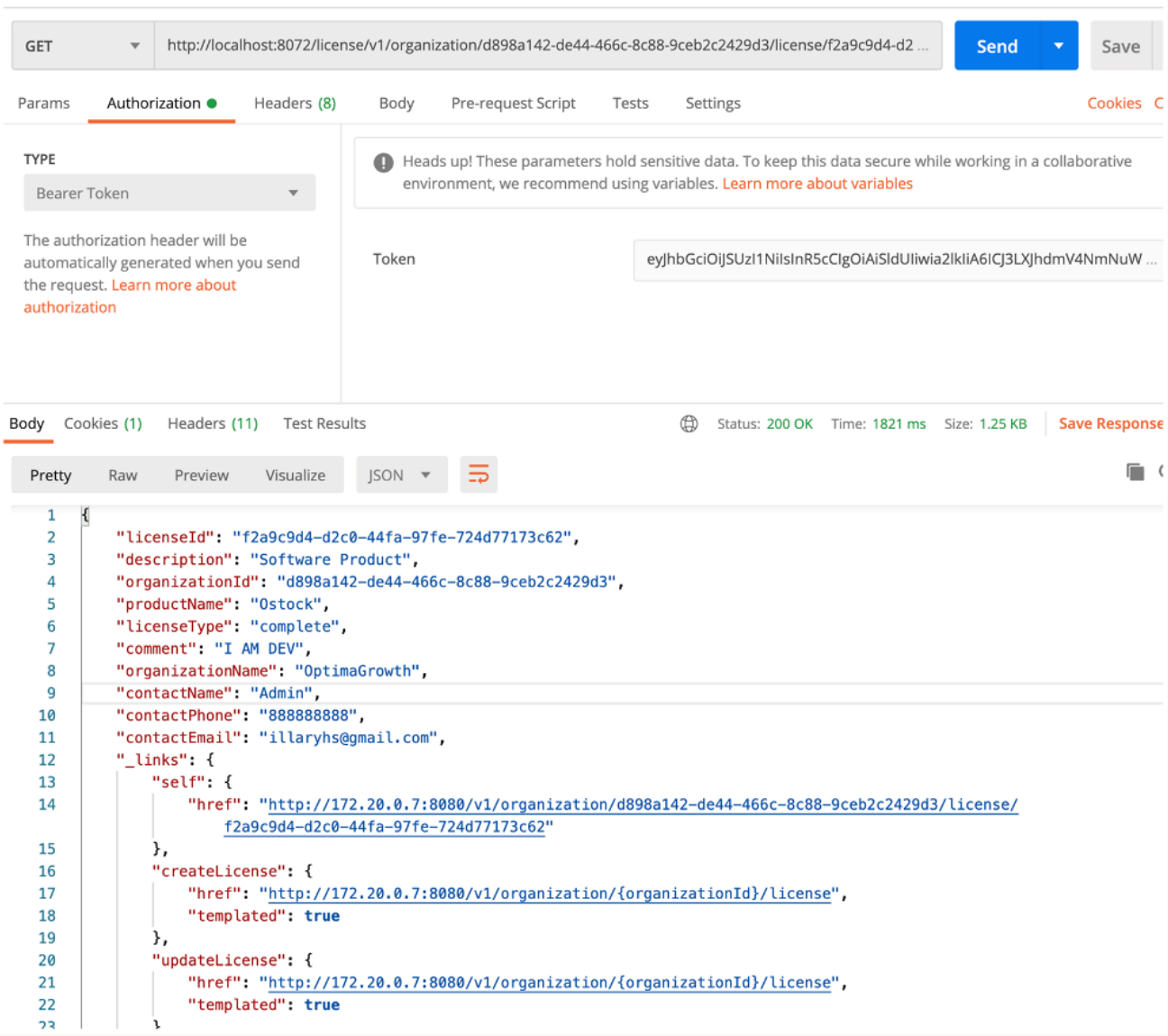


Figure 9.27 Passing the access token from the licensing service to the organization service

4.5 Parsing a custom field in a JWT

We're going to turn to our gateway for an example of how to parse a custom field in a JWT. Specifically, we'll modify the `TrackingFilter` class that we introduced in chapter 8 to decode the `preferred_username` field from the JWT flowing through the gateway. To do this, we'll pull in a JWT parser library and add it to the Gateway server's pom.xml file. Multiple token parsers are available, but we chose the Apache Commons Codec and the org.json package to parse the JSON body.

```
<dependency>
    <groupId>commons-codec</groupId>
    <artifactId>commons-codec</artifactId>
</dependency>
<dependency>
    <groupId>org.json</groupId>
    <artifactId>json</artifactId>
```

```
    <version>20190722</version>
</dependency>
```

Once the libraries are added, we can then add a new method called `getUsername()` to the tracking filter. The following listing shows this new method. You'll find the source code for this method in the class file at /gatewayserver/src/main/java/com/optimagrowth/ gateway/filters/TrackingFilter.java.

Listing 9.12 Parsing the `preferred_username` from our JWT

```
//Part of this code omitted for conciseness
private String getUsername(HttpHeaders requestHeaders){
String username = "";
if (filterUtils.getAuthToken(requestHeaders)!=null){
    String authToken =
          filterUtils.getAuthToken(requestHeaders)
          .replace("Bearer ","");                          (1)
    JSONObject jsonObj = decodeJWT(authToken);
      try {
         username =                                        (2)
         jsonObj.getString("preferred_username");
      }catch(Exception e) {logger.debug(e.getMessage());}
}
return username;
}

private JSONObject decodeJWT(String JWTToken) {
String[] split_string = JWTToken.split("\\.");
String base64EncodedBody = split_string[1];                (3)
Base64 base64Url = new Base64(true);
String body = new String(base64Url.decode(base64EncodedBody));
JSONObject jsonObj = new JSONObject(body);                 (4)
return jsonObj;
}
```

(1) Parses the token from the authorization HTTP header

(2) Pulls the preferred_username from the JWT

(3) Uses Base64 encoding to parse the token, passing in the key that signs the token

(4) Parses the JWT body into a JSON object to retrieve the preferred_username

To make this example work, we need to make sure the `AUTH_TOKEN` variable in `FilterUtils` is set to `Authorization` as in the following code snippet. (You'll find the source for this in /gatewayserver/src/main/java/com/optimagrowth/gateway/filters/FilterUtils.java.)

```
public static final String AUTH_TOKEN = "Authorization";
```

Once we implement the `getUsername()` function, we can add a `System.out.println` to the `filter()` method on the tracking filter to print the `preferred_username` parsed from our JWT that's flowing through the gateway. Now when we make a call to the gateway, we'll see the `preferred_username` in the console output.

NOTE When you make this call, you still need to set up all the HTTP form parameters, including the HTTP `Authorization` header and the JWT.

If everything was successful, you should see the following `System.out.println` in your console log:

```
tmx-correlation-id found in tracking filter:
      26f2b2b7-51f0-4574-9d84-07e563577641.
The authentication name from the token is : illary.huaylupo
```

9.5 Some closing thoughts on microservice security

While this chapter has introduced you to OpenID, OAuth2, and the Keycloak specification and how you can use Spring Cloud security with Keycloak to implement an authentication and authorization service, Keycloak is only one piece of the microservice security puzzle. As you shape your microservices for production use, you should also build your microservice security around the following practices:

- Use HTTPS/Secure Sockets Layer (SSL) for all service communications.
- Use an API gateway for all service calls.
- Provide zones for your services (for example, a public API and private API).
- Limit the attack surface of your microservices by locking down unneeded network ports.

Figure 9.28 shows how these different pieces fit together. Each of the bulleted items in the list maps to the numbers in figure 9.28. We'll examine each of the topic areas enu-

merated in the previous list and in the figure in more detail throughout the next sections.

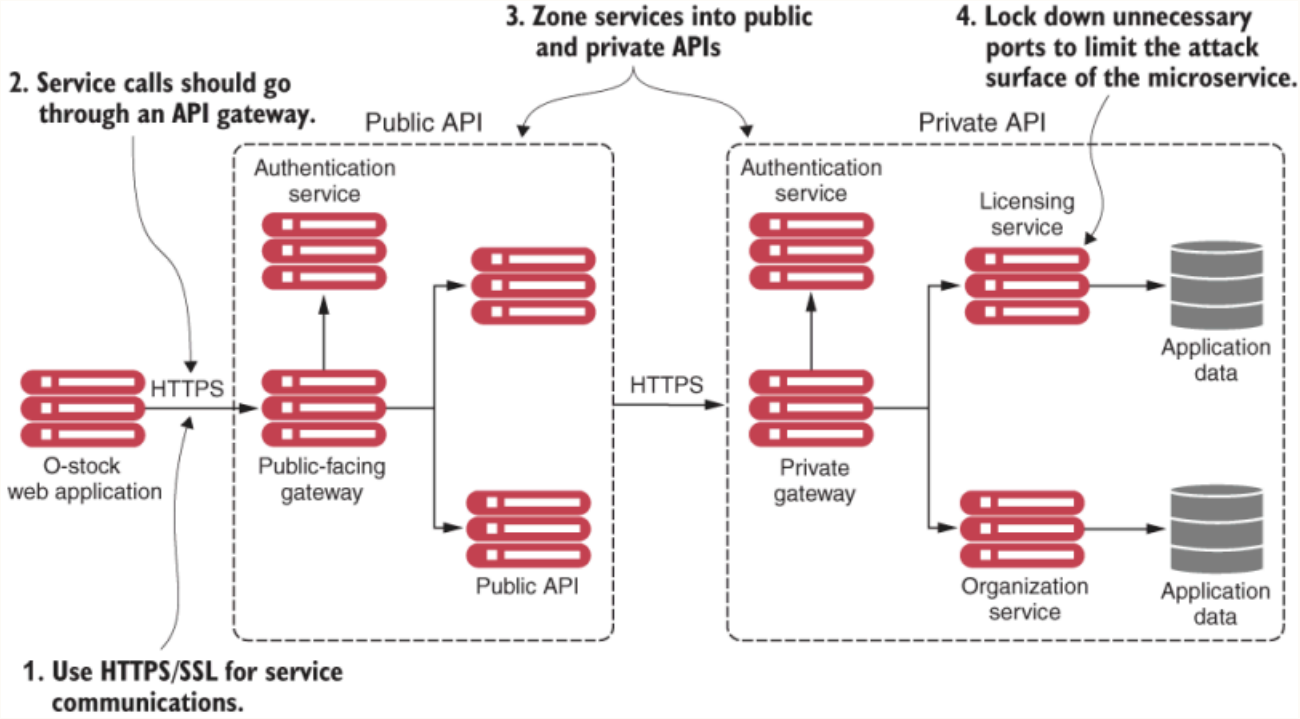


Figure 9.28 A microservice security architecture is more than implementing authentication and authorization.

5.1 Use HTTPS secure sockets layer (SSL) for all service communication

In all the code examples in this book, we've used HTTP because HTTP is a simple protocol and doesn't require setting up on every service before you start using the service. In a production environment, however, your microservices should communicate only through the encrypted channels provided through HTTPS and SSL. Note that the configuration and setup of HTTPS can be automated through your DevOps scripts.

5.2 Use a service gateway to access your microservices

The individual servers, service endpoints, and ports your services are running on should never be directly accessible to the client. Instead, use a service gateway to act as an entry point and gatekeeper for your service calls. Configure the network layer on the operating system or container that your microservices are running in to only accept traffic from the service gateway. Remember, the service gateway can act as a Policy Enforcement Point (PEP), which can be enforced in all services.

Putting service calls through a service gateway allows you to be consistent in how you're securing and auditing your services. A service gateway also allows you to lock down what port and endpoints you're going to expose to the outside world.

15.3 Zone your services into a public API and private API

Security, in general, is all about building layers of access and enforcing the concept of *least privilege*. Least privilege implies that a user should have only the bare minimum network access and privileges to do their day-to-day job. To this end, you should implement least privilege by separating your services into two distinct zones: public and private.

The public zone contains all the public APIs that will be consumed by your clients (in this book's examples, the O-stock application). Public API microservices should carry out narrow tasks that are workflow oriented. These microservices tend to be service aggregators, pulling data and carrying out tasks across multiple services. Public microservices should also be behind their own service gateway and have their own authentication service for performing authentication and authorization. Access to public services by client applications should go through a single route protected by the service gateway. Also, the public zone should have its own authentication service.

The private zone acts as a wall to protect your core application functionality and data. It should only be accessible through a single, well-known port and should be locked down to only accept network traffic from the network subnet where the private services are running. The private zone should have its own gateway and authentication service. Public API services should authenticate against the private zone's authentication service. All application data should at least be in the private zone's network subnet and only accessible by microservices residing in the private zone.

15.4 Limit the attack surface of your microservices by locking down unneeded network ports

Many developers don't take a hard look at the absolute minimum number of ports they need to open for their services to function. Configure the operating system your service is running on to only allow inbound and outbound access to the ports or a piece of infrastructure needed by your service (monitoring, log aggregation).

Don't focus only on inbound access ports. Many developers forget to lock down their outbound ports. Locking down your outbound ports can prevent data from being leaked out of your service if an attacker has compromised the service itself. Also, make sure you look at network port access in both your public and private API zones.

ummary

- OAuth2 is a token-based authorization framework that provides different mechanisms for protecting web service calls. These mechanisms are called *grants*.
- OpenID Connect (OIDC) is a layer on top of the OAuth2 framework that provides authentication and profile information about who is logged in to the application (identity).
- Keycloak is an open source identity and access management solution for microservices and applications. The main objective of Keycloak is to facilitate the protection of the services and applications with little or no code.
- Each application can have its own Keycloak application name and secret key.
- Each service must define what actions a role can take.
- Spring Cloud Security supports the JSON Web Token (JWT) specification. With JWT, we can inject custom fields into the specification.
- Securing our microservices involves more than just using authentication and authorization.
- In a production environment, we should use HTTPS to encrypt all calls between services.
- Use a service gateway to narrow the number of access points through which a service can be reached.
- Limit the attack surface for a service by limiting the number of inbound and outbound ports on the operating system that the service is running on.

Previous chapter

‹ 8 Service routing with Spring Cloud Gateway

Next chapter

10 Event-driven architecture with Spring Cloud Stream ›